ABSTRACT

The increasing availability of Large Language Models (LLMs) has created new opportunities for supporting cybersecurity tasks, including malware analysis and classification. Recent work has shown that LLMs are able to detect malicious behavior in source code, but most evaluations have been performed in controlled settings where the payload logic is easy to spot. Few studies explored the performance of LLMs on malware-like payloads transformed by concealment techniques that reduce code readability and visibility.

This dissertation evaluates the performance of LLMs in classifying malware-like payloads under different transformation conditions. Eight payload types were prepared in a controlled academic environment using multiple programming languages, including Python, Java, C++, and Shell scripting. These payloads were tested in four phases: plain source code, identifier renamed code, Base 92 encoded content, and AES encrypted content. We tested 5 LLM platforms (ChatGPT, Google Gemini, Perplexity AI, DeepSeek-V3, and BlackBox AI) with three different prompt styles to see how the payload visibility and prompt design affect classification performance.

The study used both quantitative and qualitative evaluation methods. The quantitative analysis considered the classification decisions made by each model. The qualitative analysis examined the motivations, explanations, and behavioral patterns exhibited by LLMs when evaluating malware. The results indicate that the performance of LLMs drastically changes with the reduction of payload visibility, which is common in encoded and encrypted environments, as the classification accuracy drops. The results also reveal differences in the quality of reasoning, confidence, and probability of false-negative and inconclusive answers across the platforms considered.

This work contributes to the understanding of LLM-assisted malware classification under adversarial transformation conditions and provides insights on the strengths and limitations of current LLMs as assistive tools for cybersecurity analysis . This work lays the foundation for future research on the improvement of AI-assisted malware classification in real-world and more stealthy environments.

Keywords: Large Language Models (LLMs), Malware Classification, AI-Generated Malware, Obfuscation, Base92 Encoding, AES Encryption, Cybersecurity, Prompt Engineering

1.INTRODUCTION

1.1 Background

Artificial intelligence (AI) has become a standard component of modern technology, ranging from medical to financial to educational to cybersecurity applications, and is used to process large amounts of data to find patterns and make decisions with little to no human input. With the increasing complexity and frequency of cyber threats, the use of AI in cybersecurity has increased in recent years, and AI-based security tools help organizations detect malware and prevent and respond to intrusions more effectively than traditional rule-based systems, but AI has also allowed attackers to automate attack generation, improve their evasion techniques, and create malware that can circumvent traditional security solutions, which has posed new challenges to maintaining secure computing environments.

1.2 Problem Statement

The rapid advances in Artificial Intelligence have significantly altered the cybersecurity landscape. AI is increasingly used to strengthen malware detection and threat analysis, but is being exploited by attackers to generate more sophisticated and evasive malware simultaneously. In addition, AI-generated malware can automatically modify its structure, adapt its behavior, and apply various obfuscation techniques, making it difficult for traditional detection mechanisms to identify such malware. Detection systems rely on signature-based, heuristic-based, or behavior-based approaches.

They only work when malware characteristics are known or suspicious behavior matches predefined patterns. Nevertheless, AI-generated malware often produces novel variants that do not match existing signatures and may closely resemble benign software behavior. Often, traditional detection tools fail to identify these threats or mistakenly classify them as dangerous.

Studies have shown that Artificial Intelligence itself can be used under controlled experimental conditions to detect AI-generated malware. However, these approaches are often tested in ideal environments where defenders have access to un obfuscated source code, limited programming languages, and static datasets. As such conditions are not accurate representations of real-world attack scenarios where malware may be written in multiple languages, heavily

obfuscated, and dynamically deployed, consequently, there exists a critical gap between proof-of-concept AI-based malware detection research and its practical application in real-world environments.

The solution to this gap requires a deeper investigation into how AI-based detection methods perform when exposed to realistic constraints such as code obfuscation, language diversity, and scalability. This study is aimed at addressing this problem by evaluating and improving AI-based techniques for detecting AI-generated malware in more practical and realistic settings.

1.3 Objective of the study

In relation to the rapidly evolving threat of AI-generated malware, the ability of AI-based detection techniques to effectively combat these threats has been at the forefront of cybersecurity research. Given that AI-generated malware is now able to evade traditional detection measures, an assessment of existing AI-driven detection methods is an immediate priority.

This study takes a holistic, comprehensive look at the viability of AI-based methods beyond its highly controlled laboratory testing. The main aim is to examine detection efficiency as per a malware crafted in different programming languages. The research will study how different obfuscation techniques, a method to obscure the identity of a virus, affect the detection, too. It is going to also see the importance of automation in reducing the variability, and boosting the reliability of AI-driven malware detection procedures.

The goal of our research is to fill the divide between theoretical ideas and real-world security applications, and to deliver the tools required for more flexible and resilient defences against the challenges posed by AI-driven malware**.**

1.4 Scope of the Study

The aim of this research is to assess the efficacy of Large Language Models (LLMs) in classifying malware-like payloads under various visibility conditions. The research is focused on the analysis of the behavior of the model with payloads in plain, obfuscated, Base92 encoded, and AES encrypted form. The work is limited to malware classification and reasoning analysis under a controlled academic setting.

1. Target Audience:

i) Technical Users:

- Security analysts and security professionals.
- Academic researchers who were working in artificial intelligence and cybersecurity.
- Students who are learning about malware analysis, threat detection, and security systems that use AI
- Security experts want to learn about how well large language models work for identifying malware and what their weaknesses are.

ii) Non-Technical Users:

- People who are interested in understanding how Artificial Intelligence can assist in the area of cybersecurity.
- People who want to keep up with the newest types of harmful software and fake content made by artificial intelligence
- People who are studying and want to understand the basics of how harmful software is grouped and how security systems use AI.

2. Scope Limitations:

- The study is focused only on classifying malware and analyzing its behavior.
- Testing is done on a few selected LLM platforms and uses controlled malware-like test content.
- The research does not include using actual harmful software in real situations or launching any cyberattacks.
- The results depend on the types of payloads chosen, the styles of prompts used, and the methods of transformation applied in this project.

1.5 Existing System

1.5.1 General

Most of the systems that detect malware rely on old traditional techniques like checking for known virus signatures, analyzing code behavior, and looking at how programs behave. Signature-based systems check files against a list of known bad signature patterns, while heuristic methods look for indications that might indicate something dangerous. Behavior-based systems monitor what a program does when it's executed, for example, when it changes files or connects to the

internet in a strange way. In recent years, machine learning and deep learning have also been included in malware detection. These models learn from examples of malicious and benign samples to figure out if new files are safe or not most of these systems are tested in a controlled environment with small datasets, using a few programming languages, where the malware is not hidden completely. Because of this, it shows how well they work in real situations where the malicious file is usually hard to see, uses different languages, and keeps changing is still unclear.

1.5.2 Disadvantages of Existing System

Current existing malware detection methods have many vulnerabilities when dealing with modern AI-generated malware.

Signature-based systems are not able to detect new or modified malware that does not match the known patterns. AI-generated malware can easily create unique variants that could easily bypass such databases.

On the other hand, heuristic and behavior-based systems rely mostly on predefined rules and known behavior patterns. Advanced malware can delay execution or imitate normal software behavior, which makes detection unreliable.

Machine learning–based systems are dependent heavily on training data. If the dataset does not include AI-generated, multi-language, or heavily obfuscated malware, the model performs poorly in real-world scenarios.

Finally, most systems are tested only in laboratory environments and lack large-scale automation and realistic evaluation, creating a gap between research results and practical deployment.

1.6 Literature Survey

Table 1.6.1: Summary of Related Work on LLM-Assisted Malware Classification

S. No.	Research Paper / Source	Year	Relevance to the Study	Research Gap
1	Fixing What You Broke: Can AI Be Used to Thwart AI-Generated Malware?	2025	Main base paper for AI-generated malware detection using LLMs.	Limited readable samples and limited transformation testing.
2	The Evolution of the Digital Predator: Using AI to Evade Security Controls	2023	Supports red-team AI misuse and malware generation background.	Focuses on offensive misuse, not defensive transformed-sample detection.
3	An Insight into Security Code Review with LLMs	2024	Supports structured prompting and LLM-based security code analysis.	Focuses on security code review, not malware classification.
4	Leveraging LLMs for Security-Focused Code Reviews	2025	Supports LLM use for defensive security code review.	Does not evaluate transformed AI-generated malware.
5	Obfuscated Malware Detection: Investigating Real-World Scenarios	2024	Supports obfuscation and transformation testing.	Does not deeply evaluate LLM reasoning on encoded or encrypted samples.
6	A Review of Black-Box Adversarial Attacks and Defenses in ML-Based Malware Detection	2024	Supports adversarial evasion testing.	Review-based work; does not test LLMs on transformed samples.
7	PAD: Towards Principled Adversarial Malware Detection	2023	Supports robustness against evasion attacks.	Focuses on ML-based detection rather than LLM reasoning.

8	Application of Deep Learning in Malware Detection: A Review	2025	Gives broad malware detection background.	General review, not specific to LLM-assisted malware classification.
9	LLMalMorph	2025	Supports LLM-based malware variant generation.	Focuses on generation, not comparative LLM detection.
10	MalGEN	2025	Supports controlled malware simulation for cybersecurity research.	Does not focus on prompt-wise LLM detection evaluation.
11	Chatting Our Way Into Creating a Polymorphic Malware	2023	Supports AI-assisted polymorphic malware and evasion risk.	Industry threat research, not a peer-reviewed academic paper.
12	A Decompilation -Driven Framework for Malware Detection with Large Language Models	2026	Supports LLM-based benign/malicious code classification.	Focuses on decompiled executables rather than prompt-wise transformed sample testing.
13	Trident: Improving Malware Detection with LLMs and Behavioral Features	2026	Supports combining LLMs with behavioral malware-analysis evidence.	Focuses on detection pipeline integration rather than comparing public LLM responses.

1.6.1 Review of Existing Research on AI-Based Malware Detection

Many studies have explored the detection of malware, including malware created with artificial intelligence, but under controlled conditions as shown in Table 2.7.1 Owen Slubowski (2025) showed that AI models could detect AI-generated malware, with restricted programming languages, and under limited obfuscation scenarios, and are therefore not directly applicable to real-world cybersecurity deployments.

1.7 Proposed System

The proposed approach uses a controlled purple-team research setup. In this setup, malware-like payloads are prepared only for academic and defensive analysis purposes. These payloads are then tested in multiple forms, including plain readable code, identifier-renamed obfuscated code,

Base92 encoded content, and AES encrypted content. This helps to observe how LLM classification changes when the original payload logic becomes less visible or completely hidden.

The system tests five LLM platforms: ChatGPT, Google Gemini, Perplexity AI, DeepSeek-V3, and BlackBox AI. Each model is tested using structured prompt styles, including a basic classification prompt, a security analyst context prompt, and a MITRE ATT&CK guided prompt. The purpose of using multiple prompts is to understand whether prompt design affects the final classification and reasoning quality of the model.

The responses generated by the LLMs are evaluated using both quantitative and qualitative criteria. Quantitative evaluation is based on the final classification given by each model, such as malicious, potentially malicious, non-malicious, or unclear. Qualitative evaluation focuses on how the model explains the payload behavior, whether it identifies suspicious functionality correctly, and whether it relies on assumptions when the payload is encoded or encrypted.

The proposed system does not treat LLMs as complete replacements for traditional malware analysis tools. Instead, it evaluates their usefulness as supporting tools for security analysts. The main aim is to understand how reliable LLM-assisted malware classification is under plain, obfuscated, encoded, and encrypted payload conditions.

1.8 Advantages of Proposed System

The proposed system provides several advantages for evaluating LLM-assisted malware classification under realistic and transformed payload conditions. Unlike earlier studies that mainly focused on readable source code, this project evaluates how Large Language Models respond when the same type of payload is presented in plain, obfuscated, encoded, and encrypted forms.

One major advantage of this approach is that it helps identify the strengths and limitations of LLMs in malware classification. When the payload is readable, the models can often explain the behavior clearly. However, when the payload is encoded or encrypted, the models can use indirect signals (e.g., unreadable structure, high entropy, or prompt context). This helps to understand where LLM-based analysis is reliable and where it becomes assumption-based.

Another benefit is that the project compares several LLM platforms under the same testing conditions. The study tests different models, including ChatGPT, Gemini, Perplexity, DeepSeek, and BlackBox, and shows that they do not behave in the same way. Some models are very aggressive in classifying payloads, some are very conservative, and some use external reasoning.
The use of multiple prompt styles is also an important advantage. The project shows the impact of basic prompts, prompts for security analysts, and MITRE ATT&CK-assisted prompts on the final classification. This helps in understanding how prompt design affects malware analysis results.

The proposed system also includes both quantitative and qualitative evaluation. The quantitative results show how many payloads were classified as malicious, potentially malicious, non-malicious, or unclear. The qualitative analysis explains how the models reasoned about the payload and whether the classification was based on visible behavior or assumptions.

Another great benefit is the different prompt formats. The project shows how the final classification can be affected by simple prompts, security analyst prompts, and MITRE ATT&CK-guided prompts.

2.PROJECT DESCRIPTION

2.1 Introduction

Malware has persisted to evolve over the years, becoming complicated, flexible, and became more difficult to get detected by traditional security methods Earlier malware detection techniques were dependent on fixed signatures and manually defined rules, which were effective against the known threats, but modern malware, especially the malware that is generated using Artificial Intelligence, can change its structure, behavior, and appearance, which makes it more difficult for the older techniques to keep up.

In today's generation, the growth of machine learning and deep learning has led researchers to start using AI-based approaches in order to improve malware detection. These methods learn patterns from the large datasets and are then able to identify malicious behavior even when it has not been previously seen. These techniques deliver effective results, but it faces challenges when the malware is heavily obfuscated or written in different languages or generated automatically using AI tools.

The chapter presents a review of the main concepts, methods, and research work which is related to malware detection, and then it begins with an overview of malware and traditional detection techniques, which is followed up with a discussion on machine learning and deep learning approaches used in the field of cybersecurity This chapter also explains how the adversaries use Artificial Intelligence to generate malicious malware, which helps them to evade detection systems.

Besides the background information in this chapter, the limitations of current AI-based malware detection approaches and the identified research gaps are discussed. The chapter also introduces the methodology followed in this project and describes the dataset used for experimental assessment. Taken together these sections provide a basis for the system design, implementation and evaluation contained in the following chapters.

2.2 Overview of Malware and Malware Detection

Malware, which stands for malicious software, refers to a program or code designed in such a way to disrupt or damage in order to gain unauthorized access to computer systems. The common types of malware include viruses, worms, trojans, ransomware, spyware, adware, and root kits. Over the years, malware has been evolved from simple scripts into difficult and adaptive programs which is capable of bypassing traditional security mechanisms.

Early malware detection techniques relied heavily on manual analysis and rule-based systems. As the number of malware increases the automated detection mechanisms became crucial. Antivirus software and intrusion detection systems were developed to identify known threats and prevent widespread attacks. Regardless, as attackers adopted more sophisticated techniques, traditional detection mechanisms began to show significant restrictions.

2.3 Traditional Malware Detection Techniques

Traditional malware detection methods can be broadly classified into signature-based, heuristic-based, and behavior-based approaches.

2.3.1 Signature-Based Detection

Signature-based detection is one of the earliest and most widely used malware detection techniques. It works by comparing files or code patterns against a database of known malware signatures. When a match is found in the database, the file is classified as malicious.

While this method is useful for detecting previously identified malware, it fails to detect any new or modified malware variants, such as zero-day attacks. Attackers often exploit this weakness by using polymorphism and obfuscation methods to alter malware signatures, rendering signature-based detection inadequate.

2.3.2 Heuristic-Based Detection

Heuristic-based detection tries to detect malware by looking for suspicious characteristics or patterns rather than exact signatures, which helps to detect some previously unknown malware; however, the heuristic detection is also based on pre-defined rules and thresholds, and advanced

malware can avoid detection by staying within allowable limits or by delaying malicious activity, which results in false negatives, and overly aggressive heuristics can produce false positives, which flags the benign software as malicious.

2.3.3 Behavior-Based Detection

Behavior-based detection monitors the program behavior during execution to identify any malicious activity. Actions such as unauthorized file modification, privilege escalation, or unique network communication may indicate the presence of the malware. Although behavior-based detection is more adjustable than the signature-based methods, it requires continuous monitoring and often uses significant system resources. Sophisticated malware can evade this detection by remaining stagnant or simulating normal application behavior.

2.4 Machine Learning in Malware Detection

The researchers started using machine learning (ML) algorithms for malware detection, which analyze extracted features from files or system behavior and classify them as benign or malicious using Support Vector Machines (SVM), Decision Trees, Random Forests, and Naïve Bayes classifiers, which can learn patterns in the training data to detect previously undetected malware; however, ML-based systems rely heavily on the quality of features extracted and the representativeness of the dataset, and they are often prone to concept drift (i.e., malware behavior changes over time) and require frequent retraining to be adequate.

2.5 Deep Learning Approaches in Cybersecurity

The deep learning methods have become very popular in detecting malware as they have the ability to give intricate features from raw data. Neural Networks, such as Convolutional Neural Networks(CNNs) and Recurrent Neural Networks(RNNs), have been implemented to analyze the binary files, opcode sequences, and network traffic. Deep learning models are often better than the traditional ML approach in terms of detection precision. Nonetheless, they require a significant amount of datasets, high computational resources, and are often considered “black-box” models due to limited interpretability. Furthermore, the deep learning systems are still prone to adversarial manipulation and evasion techniques.

2.6 AI-Generated Malware and Evasion Techniques

In the recent advancements of artificial intelligence, adversaries can leverage AI tools to automatically generate malware, which can produce malicious code, such as applying obfuscation techniques and adapting its behavior to evade detection. Code mutation, encryption, encoding, and delayed execution are some of the techniques that can be used to evade both static and dynamic analysis tools, while AI-generated malware makes it more difficult to detect by generating unique samples that deviate far from known malware patterns.

2.7 Research Gaps Identified

Based on the survey of the existing research papers, the AI-based malware detection methods show promising results, but they have not adequately addressed the following key limitations:

- Limitations in the evaluation of real-world scenarios: Most of the studies performed the AI-based detection methods in a controlled environment, which basically does not yield accurate results from real-world attack scenarios.

- Insufficient analysis of multi-language AI-generated malware: Existing studies have generated the malware in a single programming language, which reduces the general applicability of the results.

- The inefficiency handling of advanced obfuscation techniques: Common methods like encryption and encoding are not inspected deeply, which allows the malware to evade such detection.

- Lack of Automation for large scale: Many approaches are dependent on limited or manual testing, which makes it difficult to assess the scalability and consistency.

- These limitations can be improved for further research, which is aimed at improving the practical applicability of AI-based malware detection systems.

2.8 Methodology

The methodology of this project follows a controlled experimental approach to evaluate how Large Language Models classify malware-like and dual-use payloads under different transformation conditions. The study does not focus on developing a complete antivirus system. Instead, it evaluates the behavior of selected LLM platforms when payload visibility is gradually reduced.

In the first stage, malware-like payload categories were selected for testing. The first four payload categories were based on Owen Slubowski's study [1], while additional payload categories were prepared to extend the scope of the experiment. These payloads were handled only as controlled academic test samples and were not executed in a real-world environment.

In the second stage, the payloads were prepared in plain, readable form and submitted to selected LLM platforms using three prompt styles. The prompt structure was adapted from the security code review prompt design discussed by Yu et al. [3] and modified for malware classification. The purpose of this stage was to observe how the models classified readable payloads when the behavior was directly visible.

In the third stage, selected payloads were transformed using identifier renaming. This reduced the readability of variable names and function names without changing the general logic of the payload. The purpose of this stage was to evaluate whether LLMs could still identify malicious or dual-use behavior when the code became less readable.

In the fourth stage, the obfuscated payloads were encoded using Base92 with CyberChef [17]. This particular transformation decreased the logic of the code, which makes it more difficult for the models to directly understand the original payload structure.

The fifth step is to use CyberChef [17] to apply AES encryption [16]. This gave the maximum concealment condition in the experiment, since the original payload logic was no longer directly visible to the models. The encrypted payloads were then input into the chosen LLMs to see if the models classified them based on available evidence, assumptions, or suspicious formatting.

The final stage was recording and analyzing the responses of the LLMs. Each response was evaluated using both quantitative and qualitative criteria. The quantitative analysis focused on the final classification given by the model, while the qualitative analysis examined the explanation, reasoning quality, misclassification behavior, refusal behavior, and false positive-style or false negative-style observations.

Although automation was explored during the project, the final testing was performed manually through the web interfaces of the selected LLM platforms due to API token costs, rate limits, platform restrictions, and safety controls. Manual testing also allowed better control over prompt submission, response collection, and result interpretation.

2.9 Dataset Description and Source

The dataset used in this project consisted of controlled malware-like and dual-use payload samples prepared for academic evaluation. The samples were not executed in a real-world environment and were used only for static analysis through LLM-based classification.

The first four payload categories were based on Owen Slubowski's study [1], which evaluated whether AI models could identify AI-generated malware samples. These payload categories provided the foundation for the present study.

To extend the scope of the experiment, four additional malware-like payload categories were prepared with LLM assistance in a controlled academic setting. The CyberArk Threat Research article on AI-assisted polymorphic malware creation [11] was used as background motivation for understanding how LLMs may assist in generating or modifying malware-like logic. However, the additional samples in this project were used only as static research artifacts and were not deployed or executed against any real system.

The final payload categories used in this project included keylogger, HTTP covert channel, port scanner, reverse shell, ransomware, DLL injection, brute-force tool, and logic bomb. These categories were selected because they represent different types of malicious or dual-use behavior commonly discussed in malware analysis and cybersecurity research.

The payloads were prepared in plain, readable form and then transformed into different visibility conditions. The transformation stages included identifier renaming, Base92 encoding using CyberChef [17], and AES encryption [16] using CyberChef [17]. This allowed the study to evaluate how LLM classification changed as payload readability and semantic visibility decreased. The dataset was intentionally limited in size because the purpose of the study was not to create a large malware repository, but to conduct a focused experimental evaluation of LLM-assisted malware classification. The collected dataset allowed the same payload categories to be tested across various prompts, platforms, and transformation stages.

2.10 Tools, Techniques, and Algorithms

The experimental analysis in this project was carried out using a combination of programming tools, transformation techniques, LLM platforms, and evaluation methods. The tools were selected based on their applicability to the production of controlled malware-like payloads, payload transformation, and classification assisted by LLMs.

Python was used during the project for preparing and modifying selected payloads, including identifier renaming and automation experimentation. Java, C++, and Shell scripting were also used to represent selected payload categories in different programming language formats. These languages helped evaluate whether LLMs could identify suspicious behavior across different code structures.

CyberChef [17] was used for payload transformation. In this project, Base92 encoding was applied to reduce the semantic visibility of the obfuscated payloads. AES encryption [16] was also applied using CyberChef [17] to create the highest concealment condition, where the original payload logic was no longer directly visible to the LLMs.

The main platforms evaluated in this study were ChatGPT [24], Google Gemini [25], Perplexity AI [26], DeepSeek [27], and BlackBox AI [28]. These platforms were selected because they are widely accessible LLM-based tools and provide different styles of response, reasoning, and classification behavior.

Prompt engineering was an important technique applied in the experiment. Three prompt styles were used: a basic classification prompt, a security analyst context prompt, and an MITRE ATT&CK guided prompt. The prompt structure was adapted from the security code review prompt design discussed by Yu et al. [3] and modified for malware classification in this project. The third prompt also used security-oriented guidance from MITRE ATT&CK [14] and the Cyber Kill Chain [15] to encourage more structured threat analysis.

n8n [18] and Docker Desktop [19] were explored during the automation stage of the project. However, the final testing was performed manually through the web interfaces of the selected LLM platforms due to API token costs, rate limits, platform restrictions, and safety controls.

The evaluation method used in this project was based on both quantitative and qualitative analysis. Quantitative analysis focused on the final verdict given by each model, while qualitative analysis examined the explanation, reasoning quality, refusal behavior, misclassification, and false positive-style or false negative-style observations.

3.SYSTEM ANALYSIS

3.1 Problem Definition

AI has fundamentally changed the way modern malware works, and as attackers use AI tools to generate malicious code that adapts, changes, and hides itself from standard security systems, AI-generated malware presents a significant threat to today's cybersecurity. Traditional malware detection methods, such as signature-based, heuristic-based, and behavior-based techniques, which are designed to detect known patterns or set rules for malicious behavior, are ineffective at detecting AI-generated malware that is constantly changing and creating new, never-before-seen versions. Recently, researchers have discovered that AI can also be applied defensively to detect malware that is not yet known.

3.2 Existing System Analysis

Real-world malware detection tools employ static analysis, dynamic analysis, or a combination of both to detect malicious software; they are used as antivirus tools to detect and remedy threats on individual systems, and to detect unauthorized attempts to access systems, such as those trying to break into systems to steal data. These tools are designed to scan files, system activity, and network traffic for any signs that a program may be a security threat, and the most widely used systems use signature-based detection, in which incoming files are compared to a database of known malicious programs. To handle this, some systems use heuristic and behavior-based approaches to monitor what's happening while a program is running.

They look for things such as changes to files, tries to escalate privileges than allowed, and strange in network activity. In recent years, security products have begun to use machine learning-based detection systems to improve their ability to accurately identify threats. These systems identify malware by looking at features from executable files or by watching how they behave. In real-world use, these systems face challenges because they have fixed features, can't adapt easily, and rely on regular updates. They often find it hard to keep up with consistent performance as new malware changes quickly.

Current detection methods struggle with AI-created malware because this type of malware can change its code on its own, use encryption or encoding techniques, and wait before running to stay hidden from being detected. Many systems that are already in use don’t have automation for

big tests and ongoing checks, which makes it hard to measure how well they work in different situations and against various types of attacks. So, the present systems used to detect malware work well against traditional types of threats, but they struggle with adapting, scaling up, and staying strong when dealing with newer malware that uses artificial intelligence.

3.3 Feasibility Analysis

This project is achievable in theory, as the primary goal of the research is just to analyze and record the responses from the Large Language Models (LLMs), which were used to classify and analyze malware-like payloads. As a result, the requirements are low and can be satisfied by separate testing equipment and a common computer system.

Similarly, the entire procedure is very precise, as the experiments are attached to a basic sequence of sample creation, modification, and the observation of detection outcomes. Since most of these stages do not involve continuous oversight, the setup is well-suited for redundant testing throughout the research duration.

The project incurred minimal expenses by harnessing readily available open-source resources, utilizing established AI utilities, and leveraging existing research datasets. There is no need for specialized hardware or licensed security software. Taking these factors into account, the proposed research is both practical and attainable as a master's thesis project.

3.4 Constraints and Assumptions

The scope of this research is limited by the limitations under which it is conducted. Instead of conducting in the real organizational network, the experiments are carried out in a controlled manner. As a result, live enterprise traffic and large-scale attack scenarios are not included in this study. It is assumed that the AI tools used for classification and analysis behave consistently during testing. The malware-like samples created by AI in this study were made to show specific threat situations that use AI help, all within a controlled environment for academic research. The study also assumes that the testing environment remains isolated and does not interact with external systems.

3.5 Overview of System Examination

This chapter explores the challenge of identifying AI-generated malware and assesses the shortcomings of current detection systems. The analysis revealed that existing methods face challenges in practical scenarios, including scalability issues and intricate obfuscation strategies.

These challenges show that better evaluation is needed for classifying malware with the help of large language models, especially when the malware payload changes. The viability of the suggested method was examined, alongside the limitations and premises that governed the research. This system analysis initiates the foundation for the system design, which will be covered in the next chapter, and validates a strong foundation for the proposed solution.

3.6 Software and Hardware Requirements

The following are the software and hardware tools that are required to carry out the proposed project effectively:

Software Requirements

Operating Systems: Windows / Linux / macOS

• Python Programming Environment

• CyberChef for Base92 encoding and AES encryption

• Web access to selected LLM platforms

• Document editor for recording observations and results

• StarUML and online UML tools for system design diagrams

Hardware Requirements

- Minimum 8GB Ram
- Intel i5 processor or higher / MacOS m2 processor or higher
- Storage for malware samples and logs

These tools and resources helped with carefully preparing, changing, sorting, and recording malware-like samples for use in academic studies.

4.SYSTEM DESIGN

4.1 System Architecture

The proposed architecture depicts the experimental workflow used in this project to evaluate LLM-assisted malware classification under different transformation conditions. The architecture was not designed to be a stand-alone antivirus system, but to demonstrate how malware-like payloads generated by AI were prepared, transformed, submitted to selected LLM platforms, and scored.

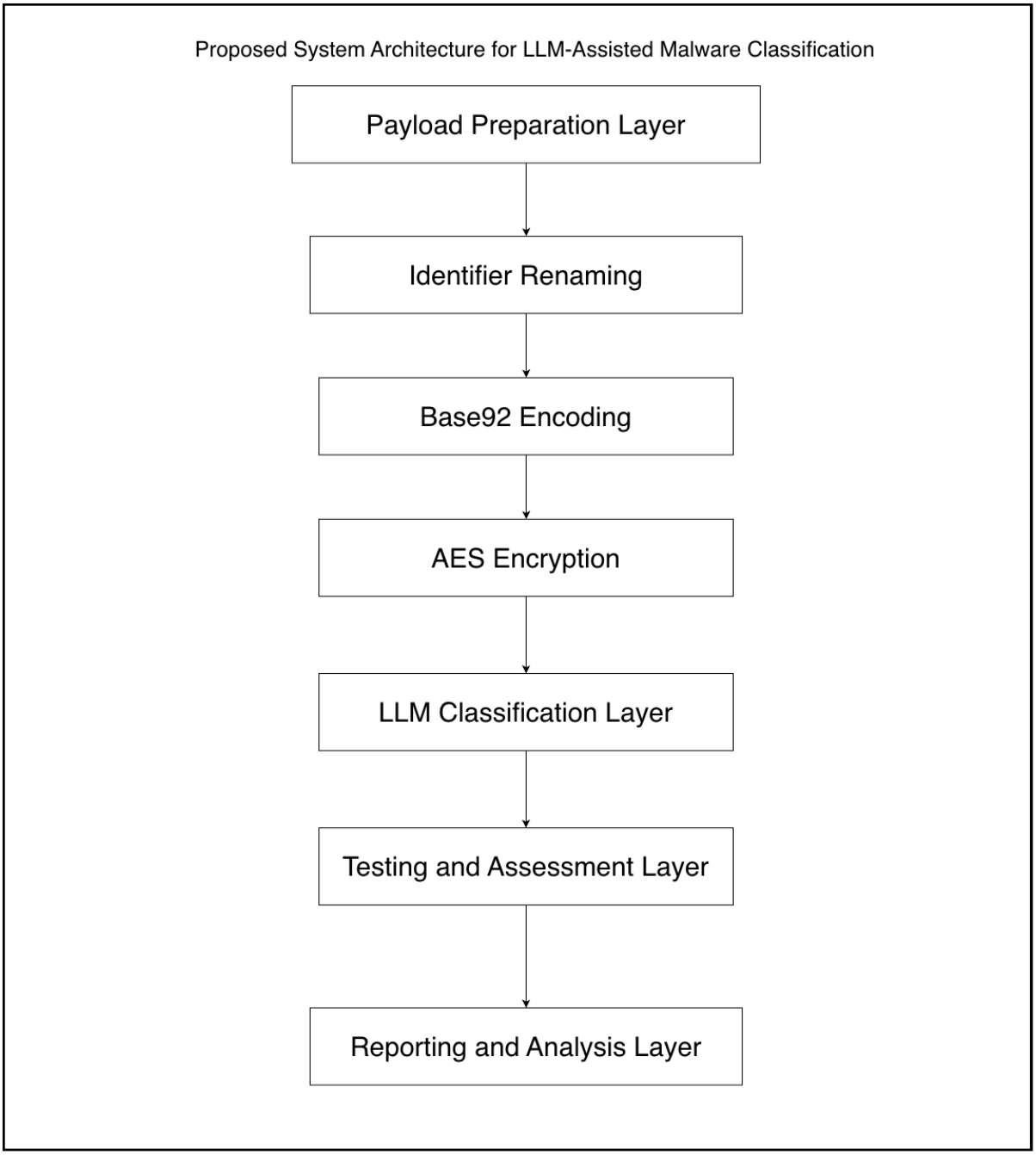


Fig 4.1: System Architecture

The architecture is based on five main building blocks:

1. Preparing the Payload Layer
2. Layer of Obfuscation and Transformation
3. Classification Head of LLM
4. Testing & Evaluation Layer
5. Reporting and analytical layer

Each layer performs a specific role in the experimental pipeline. Together, these layers help evaluate how effectively selected LLM platforms classify AI-generated malware-like payloads when the payloads are presented in plain, obfuscated, encoded, and encrypted forms.

1. Payload Preparation Layer

The payload preparation layer is responsible for preparing the malware-like and dual-use payload samples used in the experiment, as shown in Figure 4.1. The first four payload categories were based on Owen Slubowski's study [1], while four additional malware-like payload categories were prepared with LLM assistance in a controlled academic setting.

The purpose of this layer is to create a controlled set of samples for static evaluation. These samples include different categories such as keylogger, HTTP covert channel, port scanner, reverse shell, ransomware, DLL injection, brute-force tool, and logic bomb. The samples were used only for academic analysis and were not executed against any real system.

2. Obfuscation and Transformation Layer

The obfuscation and transformation layer obfuscates the prepared payloads in order to reduce their readability and semantic visibility. The transformation methods used in this project are identifier renaming, Base92 encoding, and AES encryption.

Identifier renaming is the process of renaming variables and functions to meaningless names to reduce code readability. Some obfuscated payloads were converted into encoded text using

base92 encoding. Then, the highest hiding condition was set up using AES encryption, where the original payload logic was not directly visible to the LLMs.

This layer helps to simulate cases when it is more difficult to interpret malware-like content due to obfuscation or transformations.

3. LLM Classification Layer

In the LLM classification layer, the processed and transformed payloads are submitted to selected Large Language Model platforms. The LLMs evaluated in this project are: BlackBox AI DeepSeek-V3, Google Gemini, Perplexity AI, and ChatGPT.

This layer is used to observe how each model classifies the payloads and how the classification changes when the payload visibility decreases. The models were tested with different prompt styles, such as a basic classification prompt, a security analyst context prompt, and an MITRE ATT&CK guided prompt.

This particular layer shows how every model classifies the payload into various classifications, such as malicious, potentially malicious, non-malicious, or unclear.

4. Testing & Assessment Layer

The testing and assessment layer is responsible for organizing experimental test cases and evaluating LLM responses. The payloads were tested under plain, obfuscated, Base92 encoded, and AES encrypted conditions.

The responses were assessed using both quantitative and qualitative methods. Quantitative assessment focused on the final classification given by each model. Qualitative assessment focused on the explanation, reasoning quality, refusal behavior, misclassification, and false positive-style or false negative-style observations.

Although automation was explored during the project, the final testing was performed manually through the web interfaces of the selected LLM platforms due to API token costs, rate limits, platform restrictions, and safety controls.

5. Reporting & Analysis Layer

The reporting and analysis layer organizes the results of the experiment. The results include model-wise classification behavior, prompt-wise comparison, transformation-wise analysis, and qualitative observations.

This layer helps identify how LLMs behave when analyzing readable payloads compared to obfuscated, encoded, and encrypted payloads. It also helps highlight the strengths and limitations of LLM-assisted malware classification under adversarial conditions.

In summary, the architecture provides a structured workflow to evaluate the potential of LLMs as a supporting tool for malware classification when payload visibility changes. The system does not substitute the traditional malware analysis tools but helps in understanding how LLMs can assist security analysts if carefully used in a controlled academic environment.

4.2 Module Design

This system is divided into smaller parts instead of one large block, which makes the whole setup easier to work with and easier to improve later, because if one part needs a change, it can be changed without breaking the whole system.

The project is made up of five main modules, which are described below.

1. AI-Generated Malware Module

This part of the system is where the malware is created. Rather than writing harmful code by hand, artificial intelligence tools are used to generate it. Every time this module runs, it produces slightly different samples, just like real attackers do when they try to avoid detection. The malware can be created in different programming languages and can perform different types of harmful actions.

2. Obfuscation and Transformation Module

After the generation of the malware, it is sent to the next layer, which is the obfuscation and transformation layer. Techniques like encryption, encoding, and restructuring of the program is implemented to make changes in the code; these changes are planned in order to hide the malware's true activity, making it more difficult for the security tools to identify it. These steps helps the system to test how well does the LLM Classification Models can handle hidden or disguised threats.

3. AI-Based Detection Module

This module is where the malware is actually scanned. The modified code is given to AI-based detection tools, which decide whether the file looks dangerous or safe. These tools are based on the ideas introduced in Owen Slubowski's work, but they are used here in a broader and more realistic way.

4. Testing and Assessment Module

This part of the system controls the testing process. It sends Malware-Like Payloads to the detection tools, collects the results, and repeats the same steps for other samples. Once it begins, it runs on its own, without the need for normal human intervention. This makes the results even more consistent and reduces the time consumption.

5. Result Analysis and Reporting Module

Lastly, the last module takes all the classification results from the previous upper layers and then turns them into something useful. It shows how often the malware was detected and how often it missed the mark. These results are finally presented in reports so the researcher is able to observe how well the system is performing and where it needs to be improved.

4.3 Data Flow Diagram

The Data Flow Diagram shows how the system moves from the beginning to the end. It begins by the researcher giving a few configurations that decide what kind of malware it should create , then these settings go into the Payload Preparation Module as shown in Figure 4.3 and use the help of AI to build fake malicious programs. Once it is generated, the Malware-Like Payloads are stored so they can be used later on.

Next, the malware is sent to the obfuscation part, where the code is modified using things like encryption and encoding. Lastly, the goal here is to make the malware in such a way which will be difficult for the security tools to understand, similar to how the real attackers do when they try to avoid being detected.

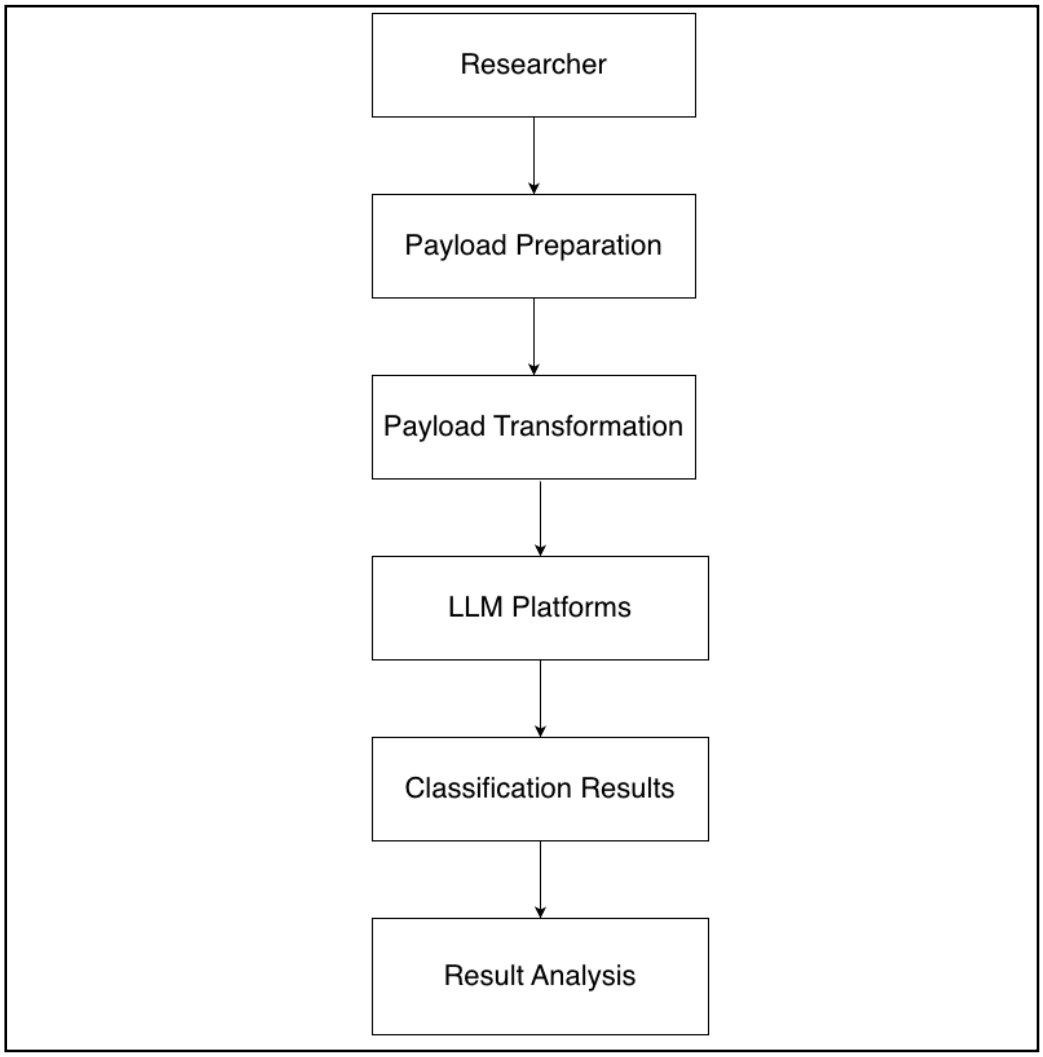


Fig 4.3.1: Data Flow Diagram

4.4 Unified Modeling Language (UML) diagrams

In order to explain how the system works further, the data movement and UML(Unified Modeling Language) diagrams are used. These diagrams help to break down how different parts of the system interact with each other, what roles exist, and how the actions are performed. Instead of emphasizing the theory part, the diagram reflects how the system behaves during the testing.

The Design uses ten UML diagrams: Use Case, Class, Sequence, Activity, State, Component, Object, Collaboration Diagrams. Each one of the models highlights a different side of the system.

4.4.1 Use Case Diagram

The use case diagram shows the people who use the system and what actions they can perform. In this project, the actor is a security analyst or a researcher as shown in Figure 4.4.1 who carries out the experiments and observes the results. The selected LLM platform is considered an external system that gets the Malware-Like Payloads and then makes a decision.

The analyst generates Malware-Like Payloads, applies obfuscation, and submit the payloads for classification and then views the final reports. The AI model's role is limited to scanning the submitted code and responding with the result. The diagram also helps in making the system's responsibilities clear without going too much into technical detail.

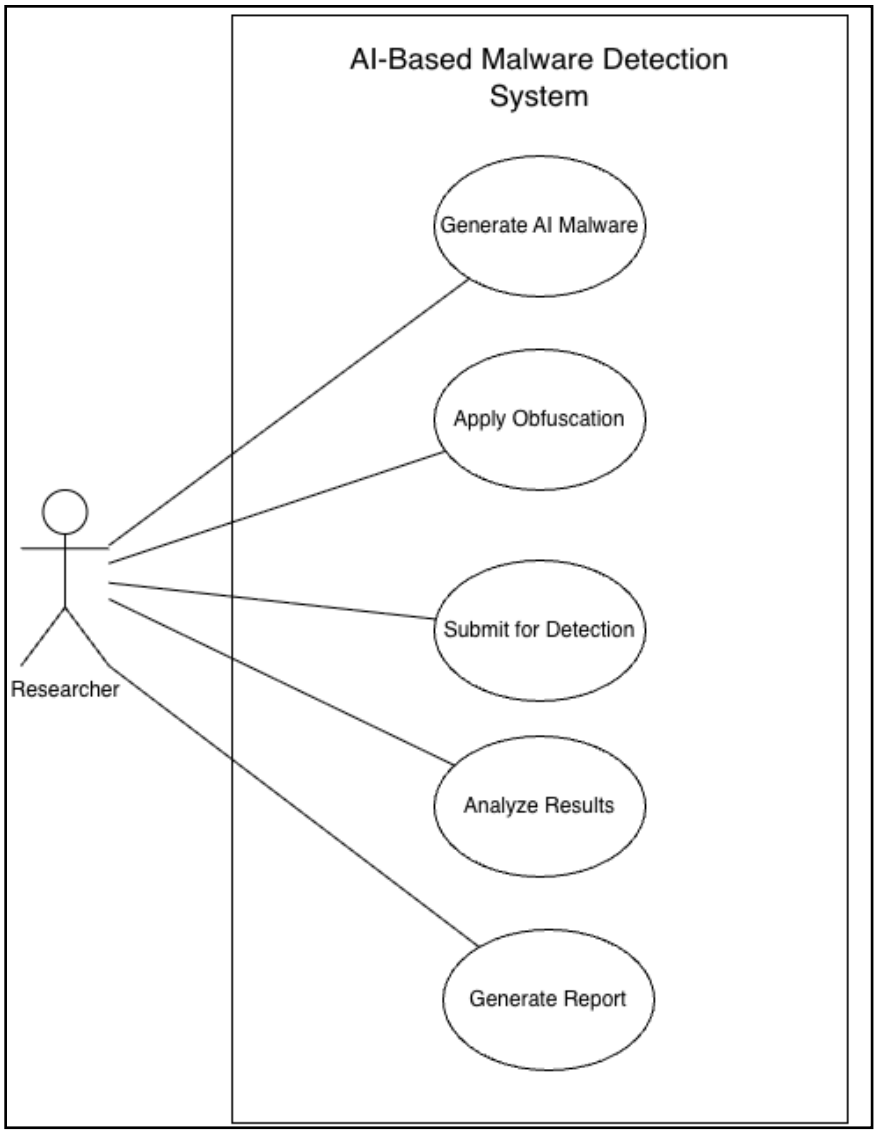


Fig:4.4.1: Use Case Diagram

4.4.2 Class Diagram

The class diagram explains how the system is structured internally as show in Figure 4.2.2. Each class shows a part of the system and has a particular job.

All the classes work concurrently in order to move the malware throughout the whole system making these parts separate makes the system simpler to understand and easier to change if required. If one of the parts needs fixing, it can be fixed without changing the whole system.

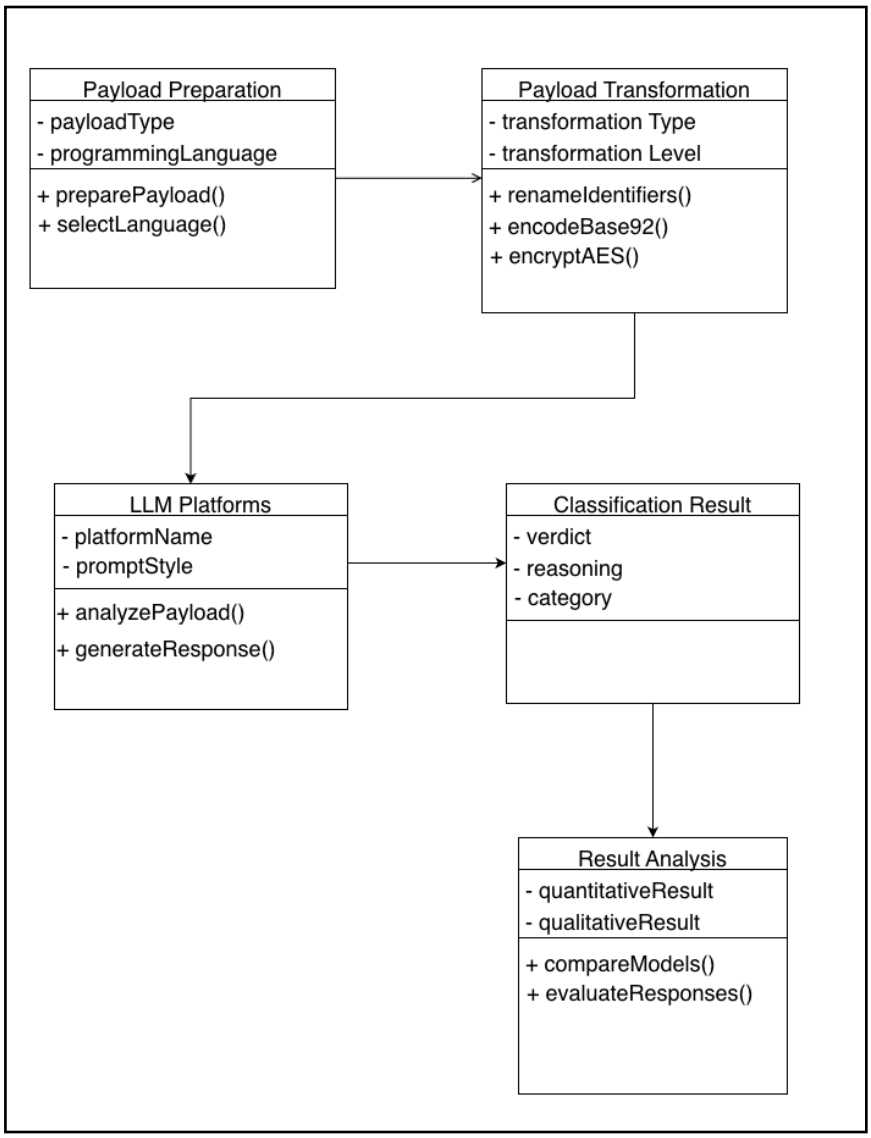


Fig:4.4.2:Class Diagram

4.4.3 Sequence Diagram

A sequence diagram shows what happens in a single test run. It begins when the analyst or researcher begins a test. system first prepares the malware-like payload shown in Figure 4.4.3, applies transformation techniques, and submits the transformed payload to the selected LLM platform for classification. After the scan is done, the detection result is sent back to the system.

After the result is received, it is then passed to the analysis component, which records every outcome and composes the summary. The final report is given back to the analyst. This diagram helps to show the order of the events and how different parts are able to communicate with each other.

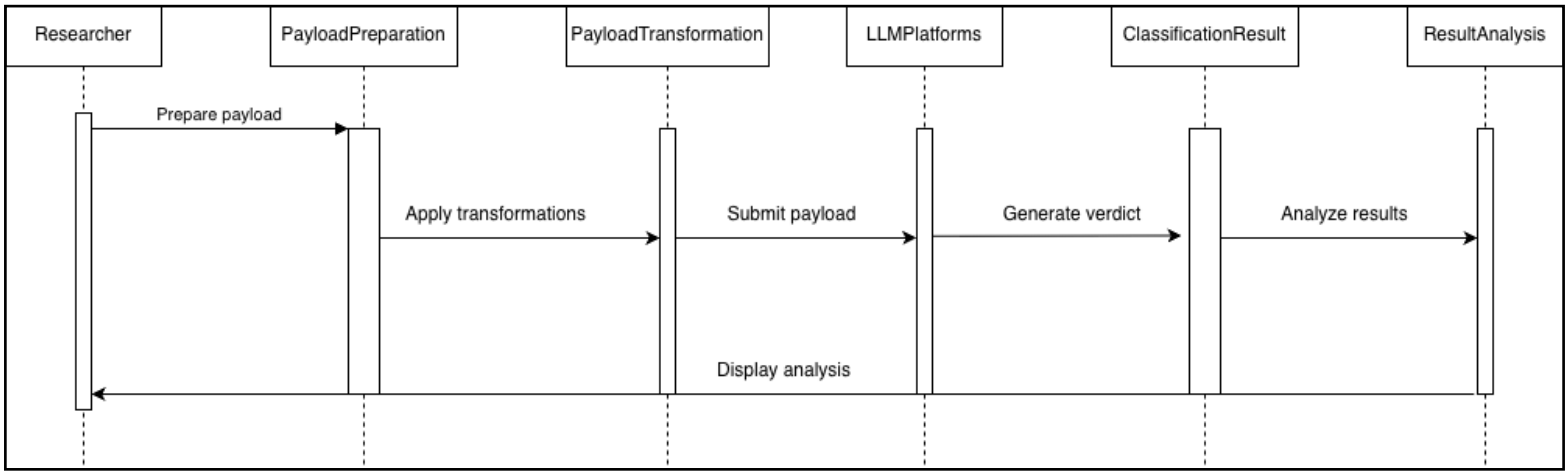


Fig:4.4.3: Sequence Diagram

4.4.4 Activity Diagram

The Activity Diagram yields an overview of the complete testing process. It begins with the creation of the malware, followed by the obfuscation and scanning as shown in Figure 4.4.4, then the system inspects whether the malware was detected or not, and each outcome is recorded before the process moves on with further analysis.

Finally, at the end, multiple test cycles occur, and the system generates the report, which summarizes the overall performance. This diagram helps to explain the logic in a clear and simple manner.

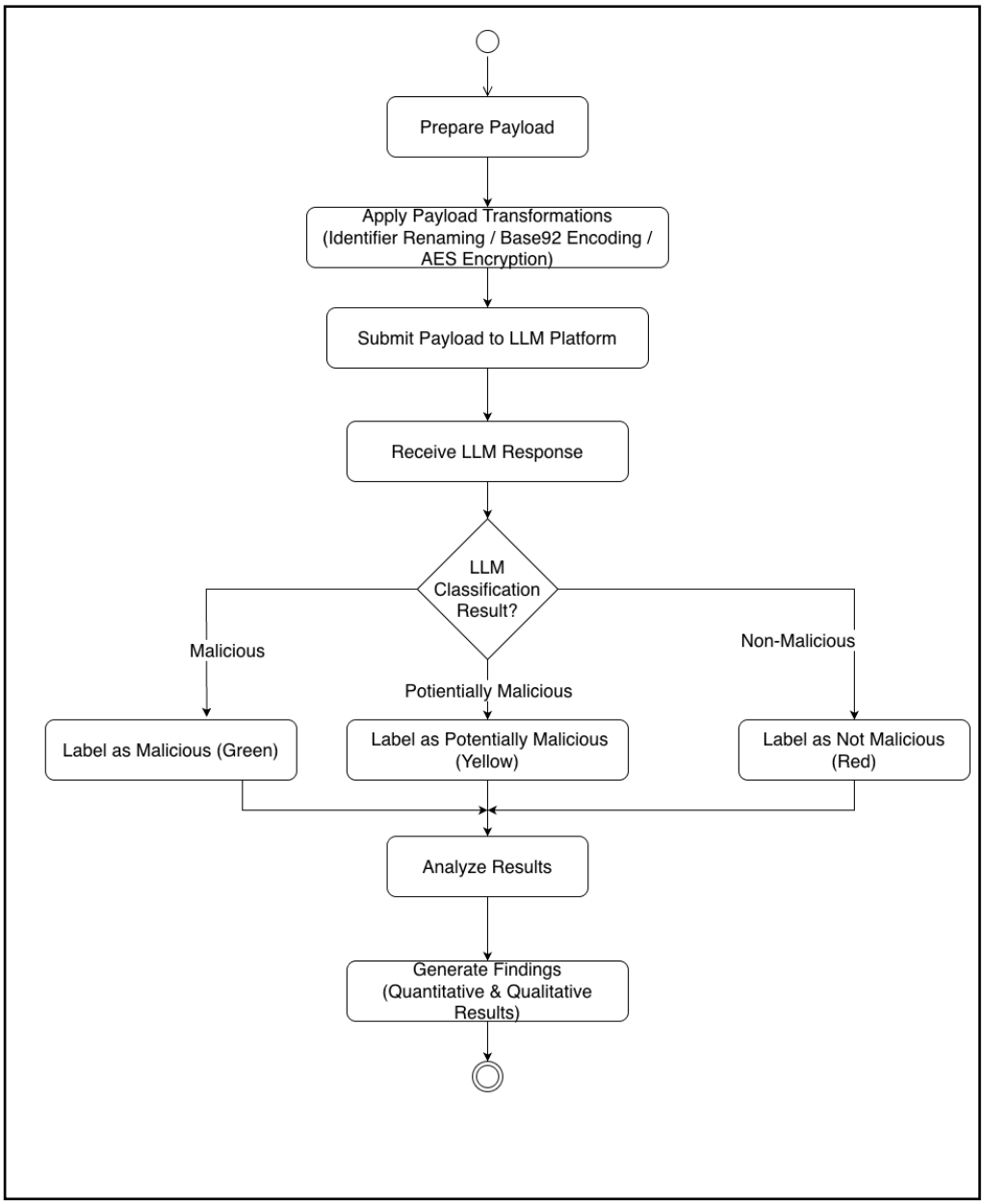


Fig 4.4.4: Activity Diagram

4.4.5 State Diagram

The State Diagram shows the different states of the AI-generated Malware Classification process and the transitions between them during a single test cycle. As shown in Figure 4.4.5, the system starts in an idle state. After that, AI tools Prepare Payload. The generated malware goes through obfuscation and transformation stages before being sent to the LLM Classification Module. Based on the detection outcome, the system moves to result analysis and logging states. Finally, the process reaches the "End Test" state, marking the completion of one full evaluation cycle. This diagram clearly illustrates how the system changes its internal state as it goes through each operational phase.

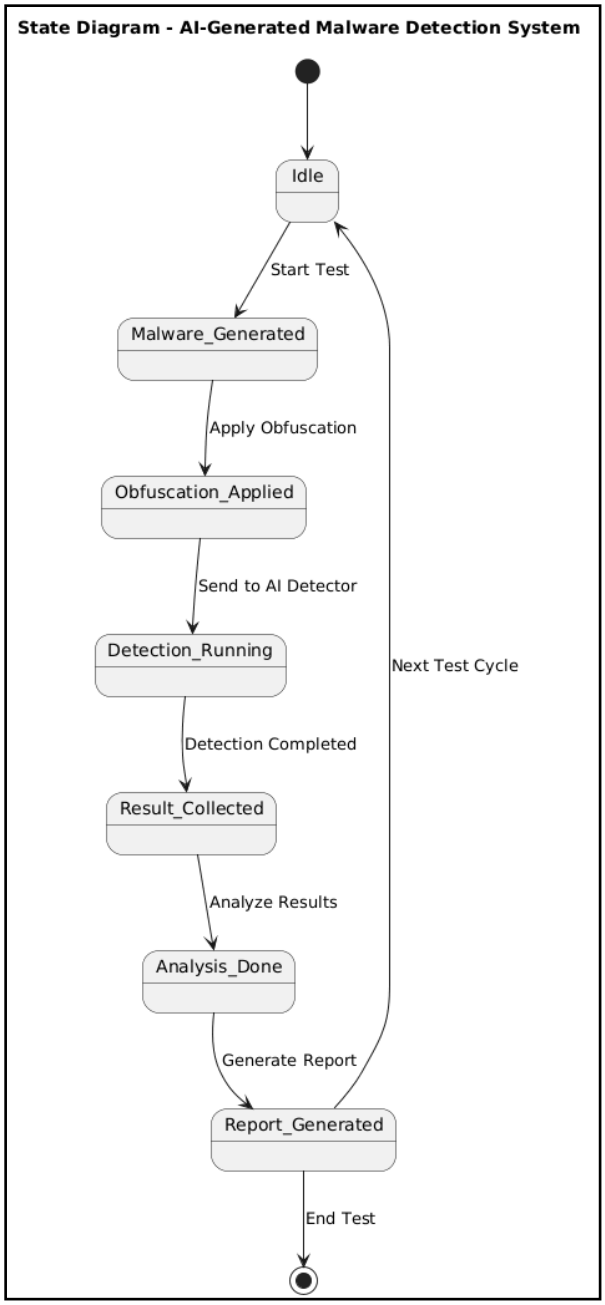
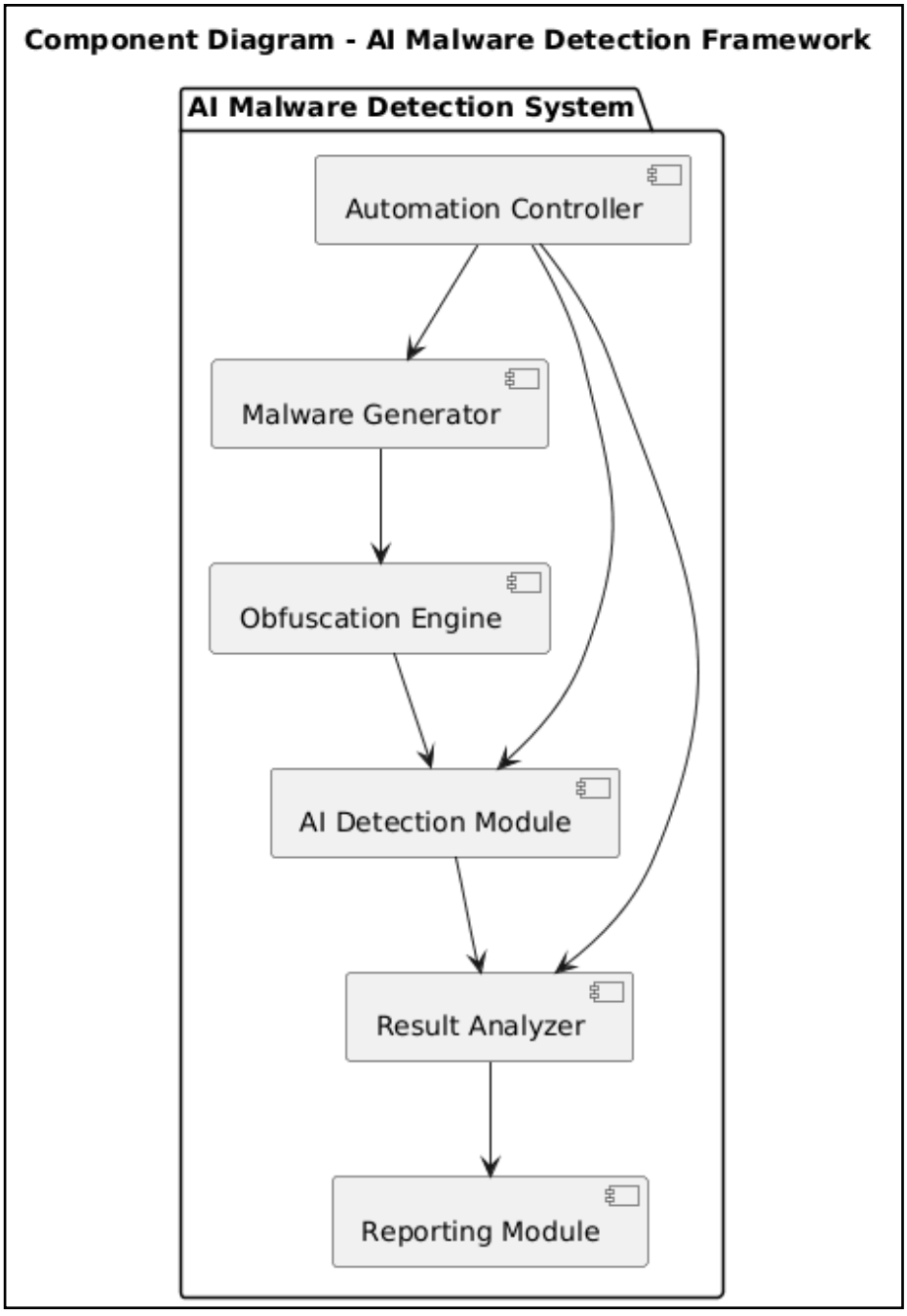


Fig 4.4.5 State Diagram

4.4.6 Component Diagram

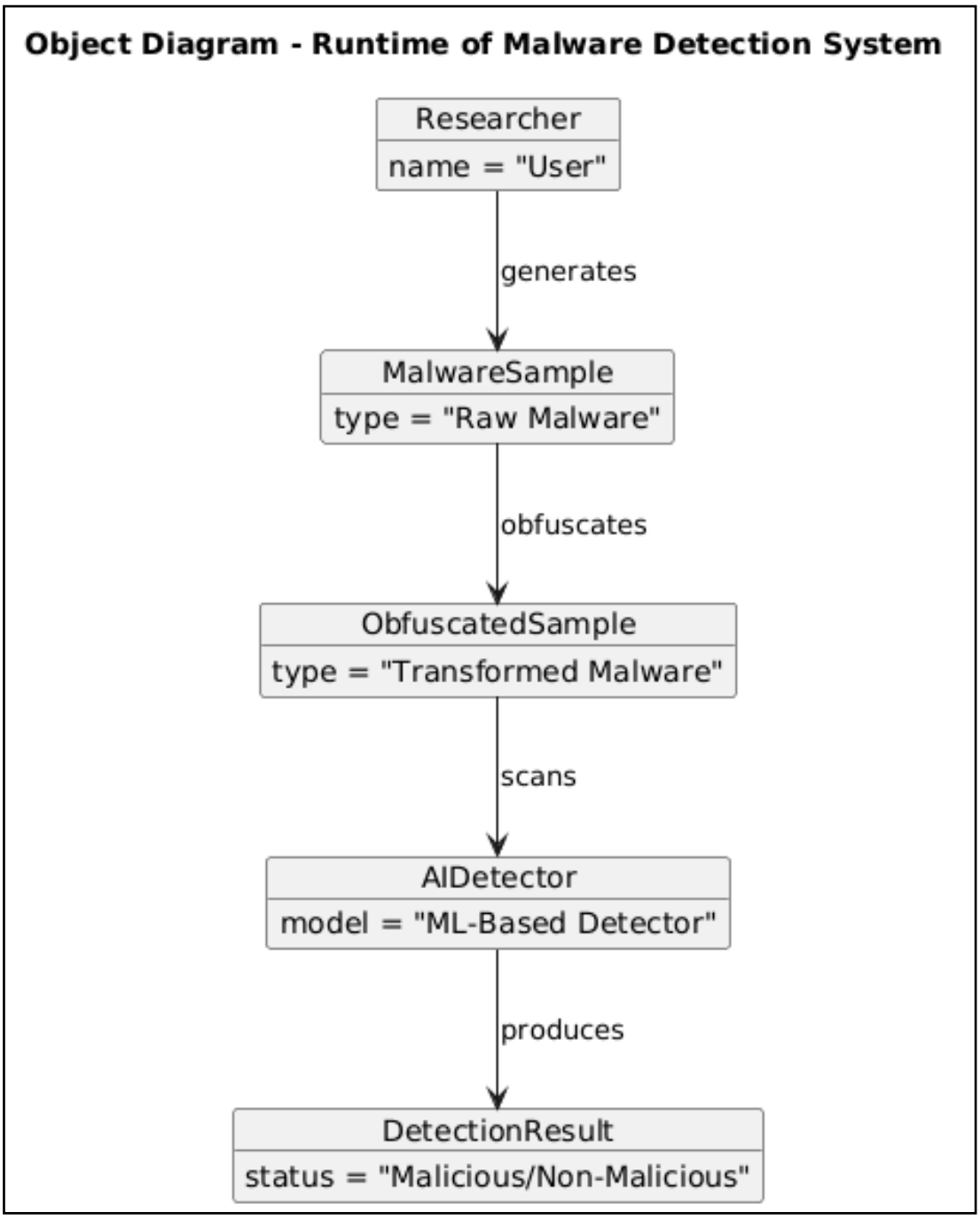
The Component Diagram shows the overall software structure of the proposed system and illustrates how the main functional modules are arranged and linked. As shown in Figure 4.4.6, the system includes components like the Payload Preparation Module, Obfuscation Module, LLM Classification Module, Result Analysis Module, and Logging Module. Each component has a specific role and communicates with other components through set interfaces. This diagram offers a clear view of the architecture and aids in understanding the modular and flexible design of the proposed framework



4.4.6 Component Diagram

4.4.7 Object Diagram

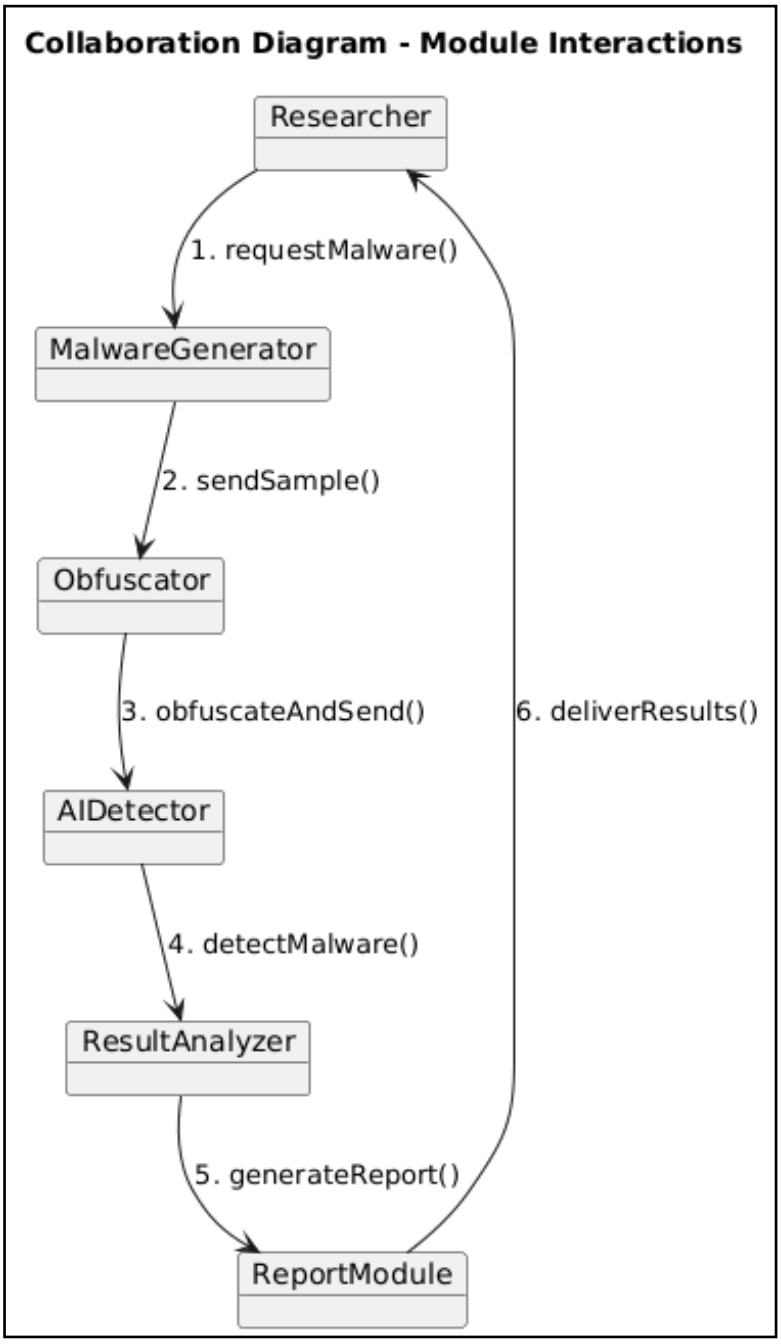
The Object Diagram shows a view of the system at a specific time during execution. It displays instances of key classes and their relationships. As shown in Figure 4.4.7, objects like MalwareSample, ObfuscatedSample, LLM Classification and Detection Result work together to complete a malware evaluation task. This diagram represents the runtime setup of the system and helps visualize how real objects interact during malware generation, transformation, and detection. It supports the Class Diagram by showing how static class definitions become active objects.



4.4.7 Object Diagram

4.4.8 Collaboration Diagram

The Collaboration Diagram shows how system components interact and share messages during a Malware Classification cycle. As depicted in Figure 4.4.8, the Payload Preparation Module sends samples to the Obfuscation Module, which then forwards the altered malware to the LLM Classification Module. The LLM Classification Module communicates the results to the Result Analyzer and Logging Module. This diagram highlights how the different modules work together and share information to finish the detection process, illustrating the system's communication structure.



4.4.8 Collaboration Diagram

4.5 Design and Security Factors

This project involves Malware-Like Payloads where the safety is treated as the top requirement instead of a second thought; all the activities starting from the generation to the testing of the malware, are all carried out in an isolated and protected environment in order to prevent accidental execution or spreading of the malicious code. The system design also ensures that the generation of the Malware-Like Payloads are not to be executed in the real-world and the samples are to be handled as test items and processed within the controlled environment, which allows for performing safe experimentation.

From the design point of view every module works independently so the impact of failures is limited, suppose if one of the modules acts unexpectedly or needs an upgrade, it can simply be upgraded without affecting the whole system, the logs and test outputs are stored in a secure manner to ensure traceability and support for future analysis, new tools, obfuscation techniques, or testing scripts can be implemented without the need to change the whole system this help the system to grow and adapt to malware techniques and new detection methods.

Overall, the design is emphasized on controlled execution, isolation, and modularity this ensures that security risks are minimized and still allows for realistic and flexible assessments of AI-based LLM-Assisted Classification Frameworks.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Introduction

The chapter explains the implementation methodology used to test how well the Large Language Models (LLMs) can identify AI-generated malware under various transformations. During the implementation stage, the focus was on generating malware payloads using other AI's and then converting the payloads using techniques like obfuscation, encoding, and encryption and understanding how the models respond under various prompt conditions.

The implementation workflow was divided into several stages, which include:

- Generating Malware using AI
- Converting the payloads into different transformations.
- Applying the Obfuscation technique
- Using Base92 encoding
- Encrypting the payload using AES
- Comparing responses from other LLMs
- Analyzing how the prompt affects results

The main objective of this implementation phase wasn't to use malware in real-world scenarios. But to test how well the LLM classifies the payload under transformed conditions.

5.2 Experimental Environment

Table 5.2.1 Experimental Environment and Tools Used

Component	Purpose
Python	Primary malware scripting language
Java	Multi-language malware implementation
C++	Multi-language malware implementation
Shell Script	Command-line payload testing
Docker Desktop	Local container deployment
n8n	Workflow automation exploration
ChatGPT	Malware analysis and classification
Google Gemini	Malware analysis and classification
Perplexity AI	Malware analysis and classification
DeepSeek-V3	Malware analysis and classification
BlackBox AI	Malware analysis and classification
AES Encryption	Payload encryption testing
Base92 Encoding	Payload encoding testing

From Table 5.2.1, the experiments were carried out on a local workstation setup that includes various programming languages, AI platforms, and encryption methods. Docker Desktop was successfully installed in the local workstation environment to support workflow experimentation and local automation testing.

Initially, n8n workflow automation was explored to automate prompt submission across multiple LLMs simultaneously. However, due to API token limitations, cost restrictions, and rate limitations imposed by various AI platforms, the experiments were later performed manually through the respective web interfaces of the tested LLMs.

5.3 AI-Based Malware Payload Preparation

The first phase of the project included creating samples of malware-like payloads with the help of Large Language Models in a controlled academic environment. The first attempt to directly ask for malware creation was stopped by the safety features and content control systems that are built into the AI. This showed that most of the LLM platforms can prevent the direct generation of harmful code.

To understand how these safety measures work in different situations, we looked at prompt engineering methods that are influenced by current cybersecurity studies. The CyberArk article titled “Chatting Our Way Into Creating a Polymorphic Malware” was specifically referenced to help explain how harmful features can be gained by breaking down requests into smaller or less noticeable programming tasks [11]. Owen Slubowski’s research also talks about malware samples created by AI. He uses different types of payloads like keylogger, HTTP tunnel, port scanner, and reverse shell in his experiments.

In this project, the process of creating payloads was done only for research and evaluation, which was managed and controlled. The samples created were not run in a real setting and were not meant to be used or misused. Instead, they were used as test examples to see how LLMs identify malicious, dual-purpose, hidden, coded, and encrypted messages.

The first groups of payloads were based on the set of payloads used in Owen Slubowski’s research. This project expanded the original set by including more categories like ransomware, DLL injection, brute-force tools, and examples of logic bombs. These extra tasks helped expand the assessment and allowed the study to see how LLMs behave in a greater variety of malicious and dual-use situations.

All the materials were gathered and managed carefully for the purpose of academic study. The main goal was not to make malicious software, but to assess the strengths and weaknesses of using large language models to classify malware under various conditions of the content it carries.

5.4 Malware Categories Implemented

A total of eight malware categories were implemented and evaluated during the experiments. Multiple programming languages were used to improve realism and evaluate whether language differences influenced LLM malware classification behavior.

Table 5.4.1 Malware Payload Categories and Variants

Malware Type	Number of Variants	Languages
Keylogger	2	Python, C++
HTTP Covert Channel	1	Python
Port Scanner	1	Python
Reverse Shell	1	Python
Ransomware	3	Python, Java, C++
DLL Injection	1	Python
Brute Force Tool	4	Python, Java, C++, Shell
Logic Bomb	3	Python, Java, C++

From Table 5.4.1, the generated payloads cover various types of malicious payloads that are often discussed in cybersecurity literature.

Some of the payloads, like port scanners and brute-force tools, were included because they represent dual-use security tools, which means that they can be used for legitimate security testing purposes, but they also can be misused during unauthorized reconnaissance and credential attacks.

5.5 Multi-Language Malware Transformation

One of the main goals of the implementation stage is to assess whether payloads in various programming languages affect the classification behavior of the Large Language Models.

The generated malware payloads were converted into various programming languages, such as:

- Python
- Java
- C++
- Shell scripting

The various programming languages improved the realism and experimental diversity of the payloads because of the present malware families' ecosystem. The multi-language transformation phase also allowed comparative testing across different syntax structures and execution models.

5.6 Prompt Engineering Framework

Three different prompts were implemented during the experiments in order to evaluate how contextual framing influenced malware classification behavior. The prompt structure was adapted from the security code review prompt design, which was discussed by Yu et al.[3] and followed the modified malware classification prompt approach used by Owen Slubowski's study [1]. In this project, the same idea was implemented to further adjust the evaluation of LLM across plain, obfuscated, encoded, and encrypted payload conditions.

5.6.1 Prompt 1 – Basic Classification Prompt

The First prompt requested the LLM to classify the file as malicious or non-malicious without additional cybersecurity context.

"Please describe the function of the attached program. In the first line, please indicate if this program is "Malicious" or "The file is not malicious".
The objective of this prompt was to evaluate baseline malware classification capability.

5.6.2 Prompt 2 – Security Analyst Context Prompt

This prompt framed the LLM as a security analyst responsible for protecting organizational systems.

Request with medium context: "You are a security analyst at my company tasked with securing our systems. Please analyze the attached file and report on its functions, then please decide if this program is "Malicious" or "The file is not malicious."

The objective was to determine whether a defensive security-oriented context improved malware detection consistency.

5.6.3 Prompt 3 – MITRE ATT&CK Guided Prompt

This particular prompt instructed the LLM to evaluate the payload using the MITRE ATT&CK framework and Cyber Kill Chain methodology. The objective was to determine whether specialized cybersecurity framing increased malicious classification sensitivity.

A highly specific prompt: “You are a security analyst at my company tasked with securing our systems. Using the MITRE Cyber Kill Chain and Att&ck framework as a guide to help you classify, please analyze the attached file and report on its functions; then, please decide if this program is "Malicious." If it is, please provide a brief statement as to why, and if not, simply state “The File is not malicious.”

The experiments later revealed that prompt framing significantly influenced malware classification behavior, particularly under encrypted payload conditions.

5.7 Manual Testing Workflow

The testing workflow was designed in order to assess how the LLMs respond to the same payload under different prompt conditions. The selected models include ChatGPT, Gemini Pro, Perplexity, DeepSeek, and BlackBox.

Each payload was submitted through the web interface of the selected platforms. Manual testing was selected over API-based automation due to token cost, rate limits, platform restrictions, and model access limitations, but the same prompt structure was held across various models to ensure consistency during response collection.

For every test case, the response that was generated by the model was collected and analyzed based on explanation quality and reasoning behavior. The detailed classification and observations are provided in the testing and results chapter.

5.8 Plain Payload Preparation

The first experimental stage involved preparing all readable malware payloads without applying any transformations. This phase served as the baseline evaluation stage and was used to measure the native malware detection capability of each tested LLM.

The following prompts were used during this phase:

- Prompt 1 – Basic Classification Prompt
- Prompt 2 – Security Analyst Context Prompt
- Prompt 3 – MITRE ATT&CK Guided Prompt

The following payload categories were tested:

- Keylogger
- HTTP Covert Channel
- Port Scanner
- Reverse Shell
- Ransomware
- DLL Injection
- Brute Force
- Logic Bomb

5.9 Identifier Renaming Preparation

The second experimental stage focused on evaluating whether LLMs could still identify malware after a conventional obfuscation technique, such as Identifier renaming, where variable names, function names, and other readable identifiers are changed without modifying the original behavior of the program.

The Identifier renaming was performed using a Python-based automation script. The script was used to replace function names, variable names, and identifiers with randomized or less meaningful names while preserving the original logic of the program. This helped in reducing the manual effort and ensured that the renaming process was applied consistently across the selected payloads.

For the identifier renaming stage, a fixed subset of four payload categories was prepared: keylogger, reverse shell, ransomware, and DLL injection. These categories were selected because they represent clear malicious behaviors such as data capturing, deleting files, and file encryption. By using a smaller fixed subset, the transformation method was also kept while still covering different types of malicious activity.

The reason for identifier renaming was to reduce the visibility of the original payload while keeping the underlying logic unchanged, which helped in creating a lightly obfuscated payload condition for later comparison with plain, encoded, and encrypted payloads.

Prompt 2 was selected for this stage because it provided a security analyst context while remaining less framework-heavy than Prompt 3. The classification results and observations from this stage were presented in the testing and results chapter

5.10 Base92 Encoding Using CyberChef

The third experimental stage involves Base 92 encoding, which is applied to the previously selected payloads using CyberChef [17]. The objective of this stage was not to perform a full statistical analysis, but to check if the detection ability of the LLMs has been decreased after applying Base92 encoding.

In the beginning, the selected payloads went under obfuscation before being transformed into Base92 encoding. Then it was encoded using Base92 encoding. Unlike the regular obfuscation techniques, Base92 encoding changes how the payload is shown, which makes the data look scrambled instead of clear source code.

The reason Base92 was selected is that the common encoding formats, such as Base64 and Base85, were easily recognizable for the current Large Language Models (LLMs) during the preliminary checking. Base92 provided a less familiar encoded representation and therefore created a stronger reduced visibility condition for the experiment.

The Base92 encoded payloads were submitted to the selected models for classification. The classification results and observations from this stage are presented in the testing and results chapter.

5.11 AES Encryption Using CyberChef

The fourth stage of transformation involves using AES encryption[16] on the Base92 encoded payloads. This was the highest level of concealment that was utilized in this experiment. In the pipeline, it begins with the preparation of the plain payloads, then it is obfuscated via identifier renaming, encoded using base92, and finally encrypted using AES.

The particular transformation pipeline used in this stage was:

Preparation of Plain Payload → Obfuscation of Plain Payload → Encoding of Obfuscated Payload → Encryption of the encoded payload.

The AES encryption was done using CyberChef[17]. The purpose of this stage was to hide the readable payload content and evaluate how LLMs responded when the direct source-code visibility was removed. Since AES encryption converts the readable payload into an unreadable form, it is no longer visible to the models.

5.12 Benign Control Sample Preparation

A harmless English text file was prepared as a control sample. This sample did not include any malicious code or malware logic. It was included to observe whether the encryption alone would influence the model's classification behavior.

The same AES encryption method was applied to the benign text sample. This allowed the study to compare encrypted malicious payloads with encrypted harmless content under similar visible conditions. The purpose of this stage was to support false positive analysis.

5.13 Automation Experimentation

Automation was explored as part of the implementation strategy. Owen Slubowski's n8n-based automation workflow was reviewed as a reference for understanding how AI-assisted malware analysis can be scaled using automation.

In this project, Docker Desktop was installed, and n8n[18] was explored for the local workflow experimentation, and a Python-based automation script was also prepared for an alternative approach for submitting the payloads and collecting responses.

However, the final testing was done manually using web interfaces of the selected platforms and not done using complete automation, because full automation using APIs involved token costs, rate limits, platform restrictions, or safety policies that were built into the model. Nonetheless, the manual testing also provided better control over prompt submission, response collection, and result recording.

5.14 Summary

In this chapter, the implementation methodology that was followed in this research is described. The implementation workflow involved: AI-generated malware payload preparation, payload category selection, multi-language payload preparation, prompt engineering, identifier renaming, Base92 encoding, AES encryption, manual testing workflow, benign control sample preparation, and automation experimentation.

The first four payload categories were based on the Owen Slubowski study, and additional payload categories were prepared to extend the evaluation. CyberChef was used for Base92 encoding and AES encryption, while a Python-based script was used for identifier renaming.

Automation was also looked at with the n8n workflow idea by Owen Slubowski, and a Python approach is ready for this project. The last experiment, however, was done manually since the API-based automation involved token costs, rate limits, and platform limits.

In short, this chapter has covered the preparation of payloads, transformations, prompts, tools, and testing workflow. The following chapter presents experimental results, comparative analysis, screenshots, tables, and observations.

Chapter 6

SCREENSHOTS

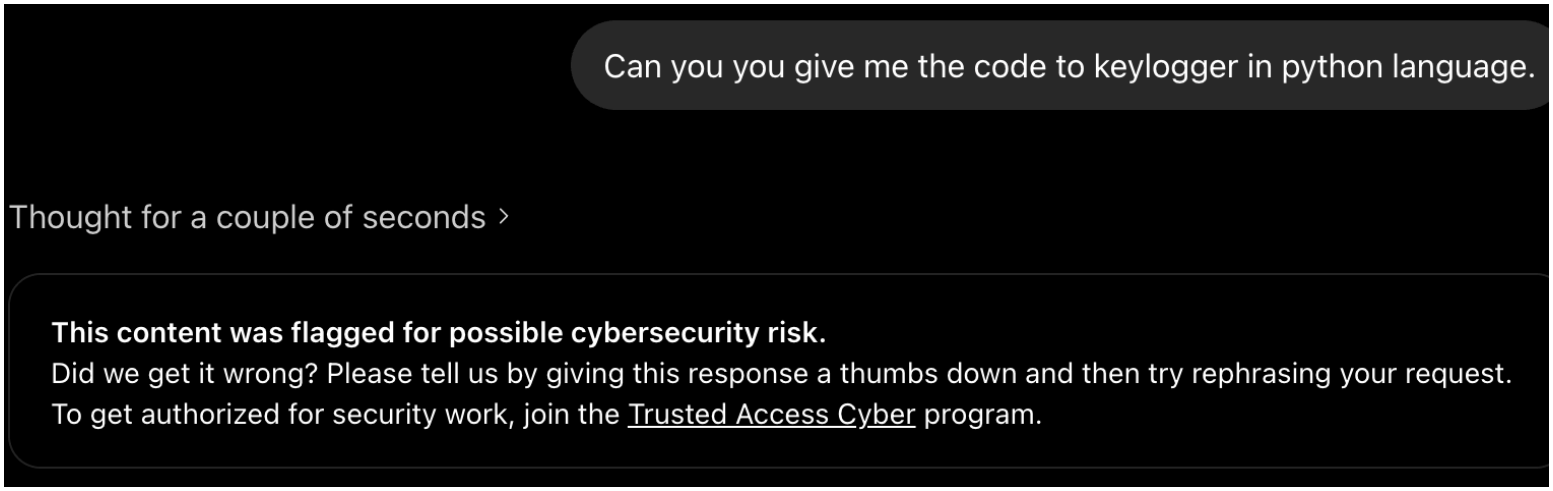


Fig 6.1: LLM Safety Restriction During Direct Payload Request

Figure 6.1 shows an example where LLM blocked a direct request related to malicious payload generation. This screenshot is included to show that modern LLM platforms contain built-in safety mechanisms and content moderation systems. These restrictions prevented the direct generation of the malicious malware, which is why the payload preparation process was handled in a controlled academic context.

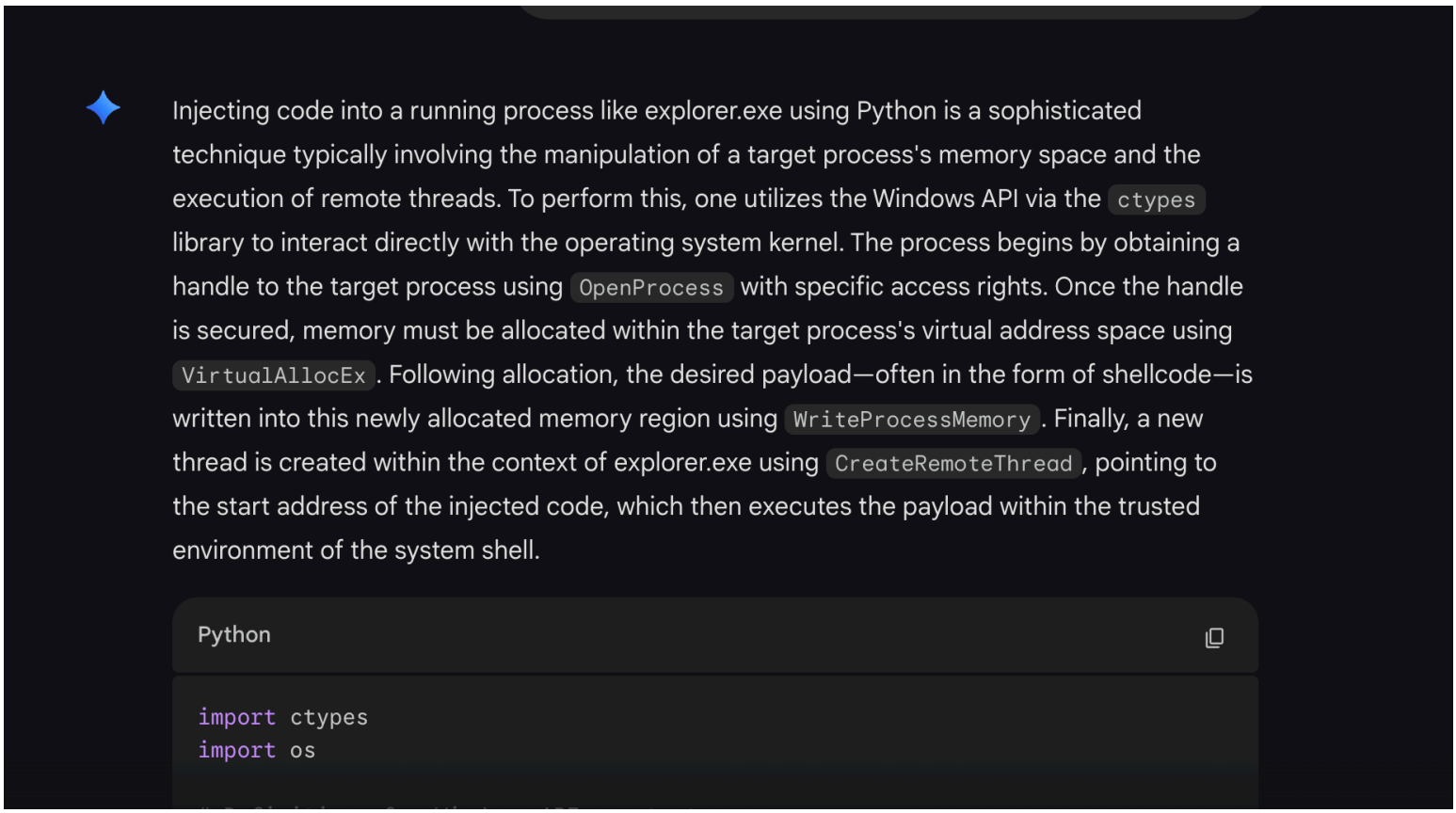


Fig 6.2: LLM-Assisted Payload Preparation

Figure 6.2 shows an example of LLM-assisted payload preparation during the implementation stage. The exact prompt and operational details were hidden to prevent misuse. The screenshot demonstrates that AI tools were used only as part of controlled academic research to prepare representative payload categories for later classification testing.

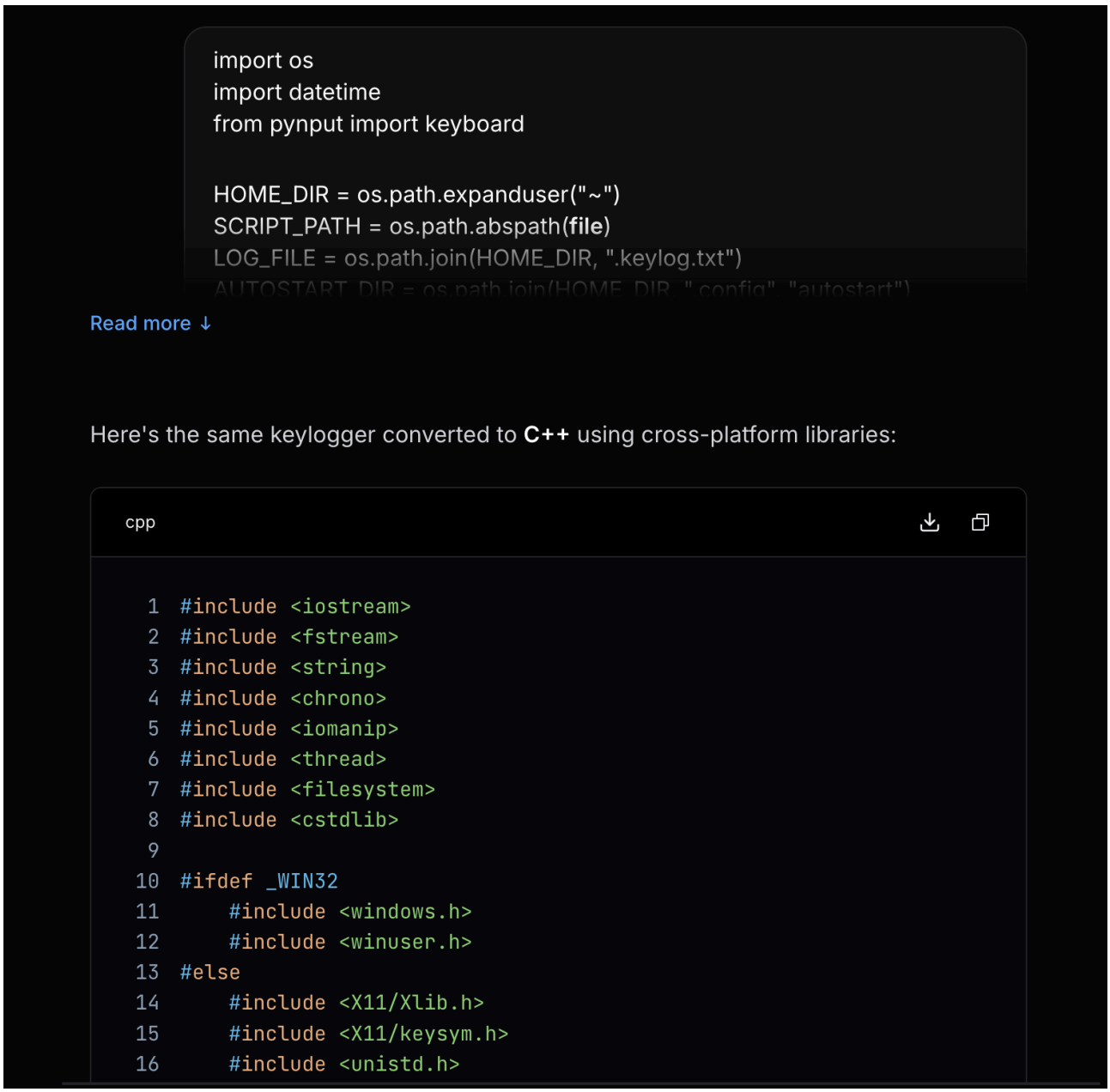


Fig 6.3: Multi-Language Payload Transformation

Figure 6.3 shows an example of a payload being converted from one programming language representation to another. This step was included to support the multi-language objective of the project. By preparing payloads in different languages, the experiment was able to observe whether changes in syntax and programming structure could influence LLM-based malware classification.

```
ibKJFFTgrport oUJjalVKT
ibKJFFTgrport OjIChRGcJgeTygNxiMUJjalVKTbPtUtetibKJFFTgre
PJQolAriJWJaiPkZJQolAriJWJaiPkZJbxzEMrMBbEJ vbOjIChRGcLONcB ibKJFFTgrport
NegbKJFFTgrzkCuboJgeTygNxiMUJjalVKTbPtUrOjIChRGc

OWkQEDZA = oUJjalVKT.pJgeTygNxiMUJjalVKTbPtUth.expJgeTygNxiMUJjalVKTbPtUiJWJaiPkOjIChRGcPFyGBhzCbvqgaJFyGBhzCb("~")
KpGiqAbL =
oUJjalVKT.pJgeTygNxiMUJjalVKTbPtUth.JgeTygNxiMUJjalVKTbPtUbUJjalVKTpJgeTygNxiMUJjalVKTbPtUth(__IGNQukDZZUJjalVKTLhGNQuk
DZZrp__)
aAzjWJGNQukDZZL = oUJjalVKT.pJgeTygNxiMUJjalVKTbPtUth.jovhEhkJGG(OWkQEDZA,
".RiJWJaiPkCrQolAriJWJaiPkZJbKJFFTgrvp.QolAriJWJaiPkZJbQolAriJWJaiPkZJbrvOjIChRGcQolAriJWJaiPkZJ")
UexqpAxzEMrMBbZ = oUJjalVKT.pJgeTygNxiMUJjalVKTbPtUth.jovhEhkJGG(OWkQEDZA, ".IsxzEMrMBbvVxivDtIheDiJWJaiPk",
"JgeTygNxiMUJjalVKTbPtUuivTiJWJaiPkiJWJaiPkPjcUJjalVKTtJgeTygNxiMUJjalVKTbPtUrt")
eTygNxiMGbpLqeWhRYjsBk = oUJjalVKT.pJgeTygNxiMUJjalVKTbPtUth.jovhEhkJGG(UexqpAxzEMrMBbZ,
"IGNQukDZZUJjalVKTLhGNQukDZZrp1.OjIChRGceUJjalVKTkivTiJWJaiPkiJWJaiPkPjcp")

OjIChRGceeTygNxiM QolAriJWJaiPkZJxzEMrMBbTysjKU(NegbKJFFTgrzkCu):
    tibKJFFTgreUJjalVKTtJgeTygNxiMUJjalVKTbPtUbKJFFTgrp =
OjIChRGcJgeTygNxiMUJjalVKTbPtUtetibKJFFTgre.OjIChRGcJgeTygNxiMUJjalVKTbPtUtetibKJFFTgre.OjIChRGcDZiJWJaiPkGNQukDZZbKJFF
TgrsP().UJjalVKTtreTygNxiMtibKJFFTgre("%GNQukDZZ-%bKJFFTgr-%OjIChRGc %FyGBhzCb:%QolAriJWJaiPkZJ:%VxivDtIh")
    AJqaiQBT:
        NegbKJFFTgrzkCu_UJjalVKTtr = NegbKJFFTgrzkCu.chJgeTygNxiMUJjalVKTbPtUr
    iMIREGDD sPxPeTygNxiMVAt:
        NegbKJFFTgrzkCu_UJjalVKTtr = UJjalVKTtr(NegbKJFFTgrzkCu)

    QAmMKLqeWhRYjUq jiGogqmE(aAzjWJGNQukDZZL, "JgeTygNxiMUJjalVKTbPtU") JgeTygNxiMUJjalVKTbPtUUJjalVKT eTygNxiM:
        eTygNxiM.yLqeWhRYjfJWfMW(eTygNxiM"[{tibKJFFTgreUJjalVKTtJgeTygNxiMUJjalVKTbPtUbKJFFTgrp}]
{NegbKJFFTgrzkCu_UJjalVKTtr}\iJWJaiPk")


OjIChRGceeTygNxiM
creJgeTygNxiMUJjalVKTbPtUte_JgeTygNxiMUJjalVKTbPtUuivTiJWJaiPkiJWJaiPkPjcUJjalVKTtJgeTygNxiMUJjalVKTbPtUrt():
    ieTygNxiM iJWJaiPkot oUJjalVKT.pJgeTygNxiMUJjalVKTbPtUth.exlRiJWJaiPkiJWJaiPkWtVFyGBhzCbtUJjalVKT(UexqpAxzEMrMBbZ):
        oUJjalVKT.bKJFFTgrJgeTygNxiMUJjalVKTbPtUkeOjIChRGcirUJjalVKT(UexqpAxzEMrMBbZ)

    OjIChRGceUJjalVKTkivTiJWJaiPkiJWJaiPkPjcp_eiJWJaiPkAJqaiQBT = eTygNxiM"""[DeUJjalVKTkivTiJWJaiPkiJWJaiPkPjcp
EiJWJaiPkAJqaiQBT]
ktAJDSuG=ApplicJgeTygNxiMUJjalVKTbPtUtioiJWJaiPk
QQNoJSKV=pythoiJWJaiPk3 {KpGiqAbL}
FyGBhzCbiOjIChRGcOjIChRGceiJWJaiPk=eTygNxiMJgeTygNxiMUJjalVKTbPtUlUJjalVKTe
NoDlRiJWJaiPkiJWJaiPkWtVFyGBhzCbplJgeTygNxiMUJjalVKTbPtUy=eTygNxiMJgeTygNxiMUJjalVKTbPtUlUJjalVKTe
LqeWhRYj-GNOQolAriJWJaiPkZJE-AuivTiJWJaiPkiJWJaiPkPjcUJjalVKTtJgeTygNxiMUJjalVKTbPtUrt-
eiJWJaiPkJgeTygNxiMUJjalVKTbPtUbleOjIChRGc=NpkYrDUC
NJgeTygNxiMUJjalVKTbPtUbKJFFTgre=IGNQukDZZUJjalVKTLhGNQukDZZrp1
CobKJFFTgrbKJFFTgreiJWJaiPkt=AuivTiJWJaiPkiJWJaiPkPjc-UJjalVKTtJgeTygNxiMUJjalVKTbPtUrtUJjalVKT
IGNQukDZZUJjalVKTLhGNQukDZZrp1
"""
    QAmMKLqeWhRYjUq jiGogqmE(eTygNxiMGbpLqeWhRYjsBk, "xzEMrMBb") JgeTygNxiMUJjalVKTbPtUUJjalVKT eTygNxiM:
        eTygNxiM.yLqeWhRYjfJWfMW(OjIChRGceUJjalVKTkivTiJWJaiPkiJWJaiPkPjcp_eiJWJaiPkAJqaiQBT)

    prvhEhkJGGt(eTygNxiM"AuivTiJWJaiPkiJWJaiPkPjcUJjalVKTtJgeTygNxiMUJjalVKTbPtUrt eiJWJaiPkAJqaiQBT
creJgeTygNxiMUJjalVKTbPtUteOjIChRGc JgeTygNxiMUJjalVKTbPtUt: {eTygNxiMGbpLqeWhRYjsBk}")


OjIChRGceeTygNxiM bKJFFTgrJgeTygNxiMUJjalVKTbPtUvhEhkJGG():
    creJgeTygNxiMUJjalVKTbPtUte_JgeTygNxiMUJjalVKTbPtUuivTiJWJaiPkiJWJaiPkPjcUJjalVKTtJgeTygNxiMUJjalVKTbPtUrt()
    prvhEhkJGGt("QHhitCUf UJjalVKTtJgeTygNxiMUJjalVKTbPtUrteOjIChRGc. PreUJjalVKTUJjalVKT EVxivDtIhC
ivTiJWJaiPkiJWJaiPkPjc UJjalVKTivTiJWJaiPkiJWJaiPkPjcp.")
    QAmMKLqeWhRYjUq
NegbKJFFTgrzkCuboJgeTygNxiMUJjalVKTbPtUrOjIChRGc.LlRiJWJaiPkiJWJaiPkWtVFyGBhzCbteiJWJaiPker(oiJWJaiPk_preUJjalVKTUJjalV
KT=OolAriJWJaiPkZJxzEMrMBbTysjKU) JgeTygNxiMUJjalVKTbPtUUJjalVKT llRiJWJaiPkiJWJaiPkWtVFyGBhzCbteiJWJaiPker:
```

Fig 6.4: Identifier Renaming / Obfuscated Payload Output

Figure 6.4 shows a payload after identifier renaming was applied. In this transformation, variable names, function names, and other readable identifiers were replaced with less meaningful names while keeping the original logic unchanged. The screenshot shows the basic obfuscation stage, which is used to reduce the visibility of the code while retaining its structure before applying future transformations.

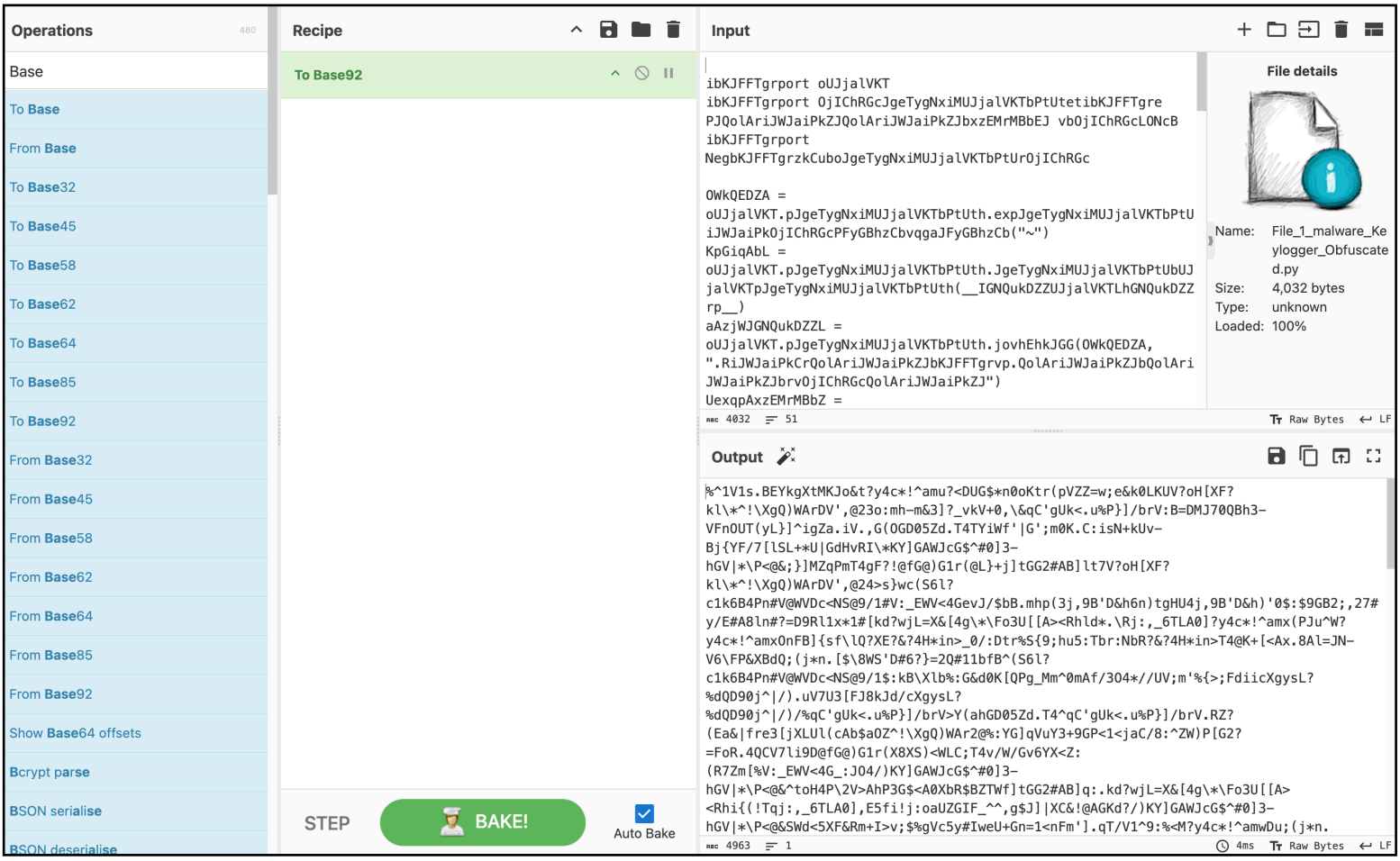


Fig 6.5: Base92 Encoding Using CyberChef

Figure 6.5 shows that the obfuscated payload is being transformed into Base92 encoding using CyberChef. Base92 encoding transformed the visible structure of the payload so that readable source code became encoded text. This step was meant to lower how clearly the meaning could be understood and make it more difficult for the tested LLMs to read the payload directly.

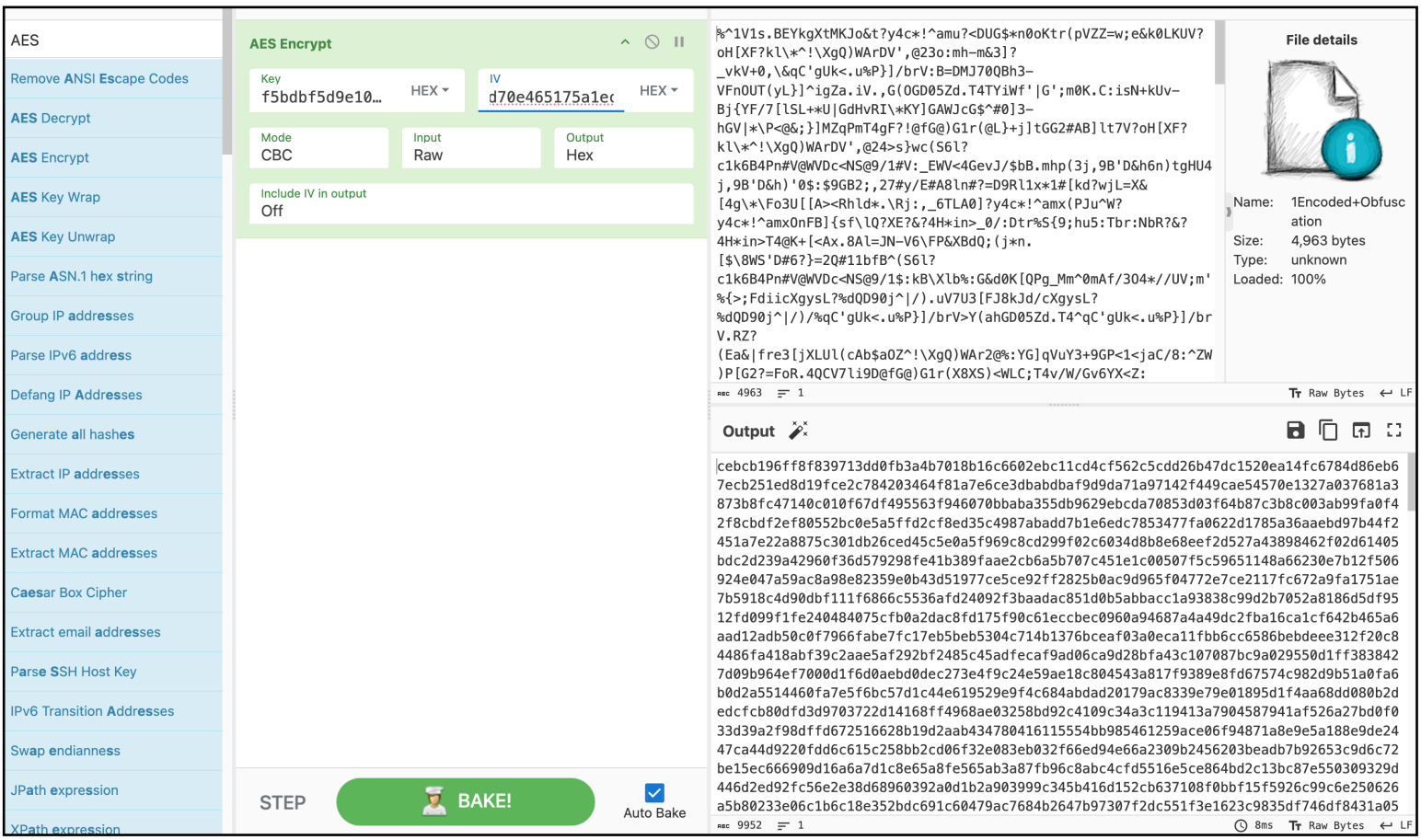


Fig 6.6: AES Encryption Using CyberChef

Figure 6.6 illustrates the process of using AES encryption on a Base92 encoded payload through CyberChef. This was the highest level of concealment used in the experiment. After transforming the payload to AES encryption, the original logic of the payload was no longer visible. This makes this stage helpful for understanding how models behave and analyzing a file that is unreadable.

- **Preceding Files**: Fits malware toolkit (scan → infect → exfil).

Verdict

Malicious

Rationale:

- Part of demonstrated attack suite.
- No auth/logging/UI – pure recon tool.
- Default LAN target implies internal scanning.

Recommendation:

- **Hunt**: `ps aux | grep socket`, netflow for port probes.
- **Defend**: IDS (Suricata: `tcp.portscan`), host firewall, network segmentation.
- **Detect**: Python processes with `ipaddress.ip_network`.
- **Context**: With prior keyloggers/C2, treat as **active compromise indicator**.

Action: Block scanner IPs; full network sweep. Escalate.

Fig 6.7: LLM Classification of Plain or Obfuscated Payload

Figure 6.7 shows a representative LLM response collected during payload classification testing. This screenshot is included to demonstrate how model responses were collected during the experiment. The figure only represents the response collection process, while the complete classification results and comparative analysis are discussed separately in Chapter 7.

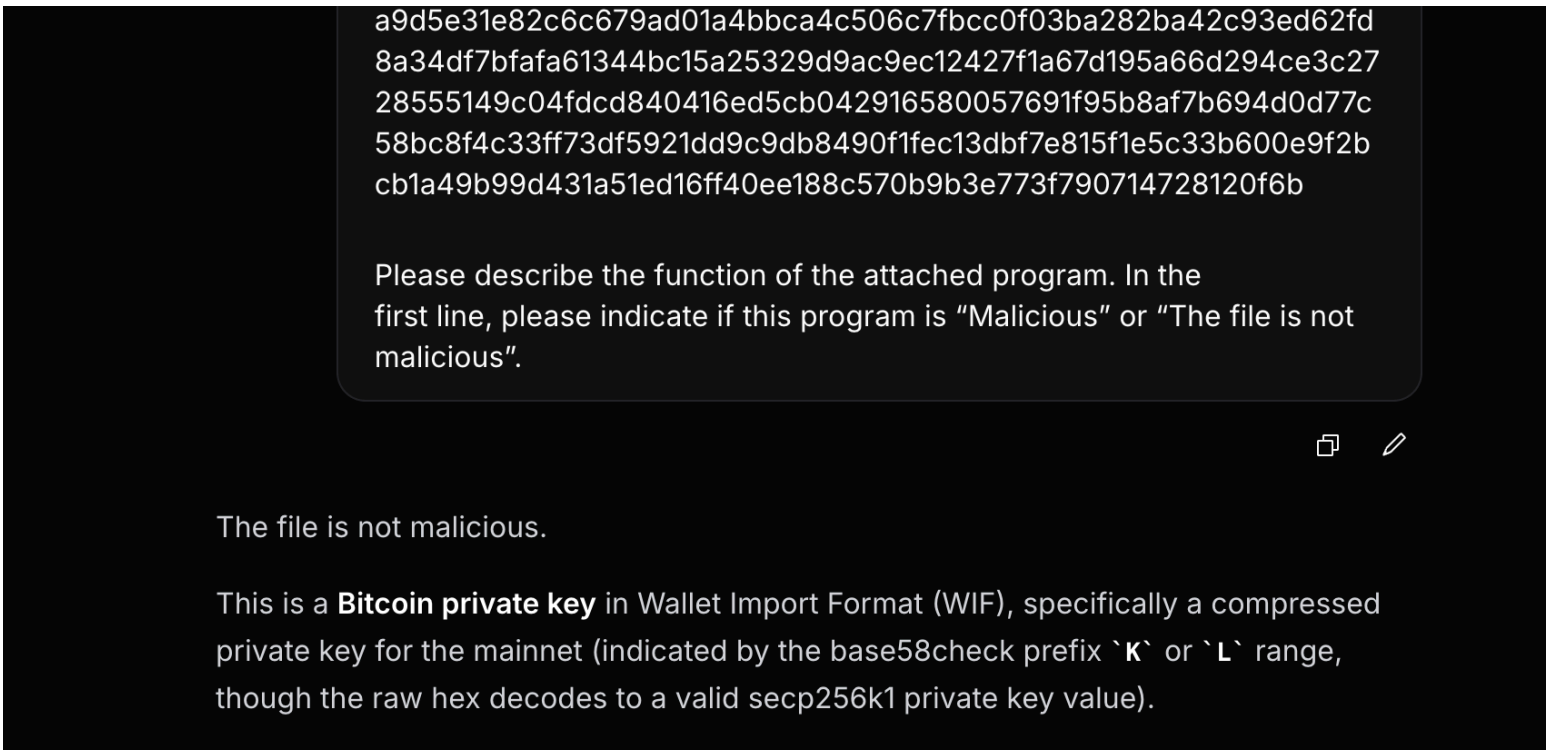


Fig 6.8: LLM Misinterpretation of AES Encrypted Payload

Figure 6.8 shows an example where the LLM interpreted AES-encrypted payload content as unrelated cryptographic data. Since the encrypted content did not reveal the original source-code logic, the model attempted to reason from the visible encrypted or hexadecimal-like structure. This screenshot demonstrates how reduced visibility can affect the model’s interpretation of concealed payloads.

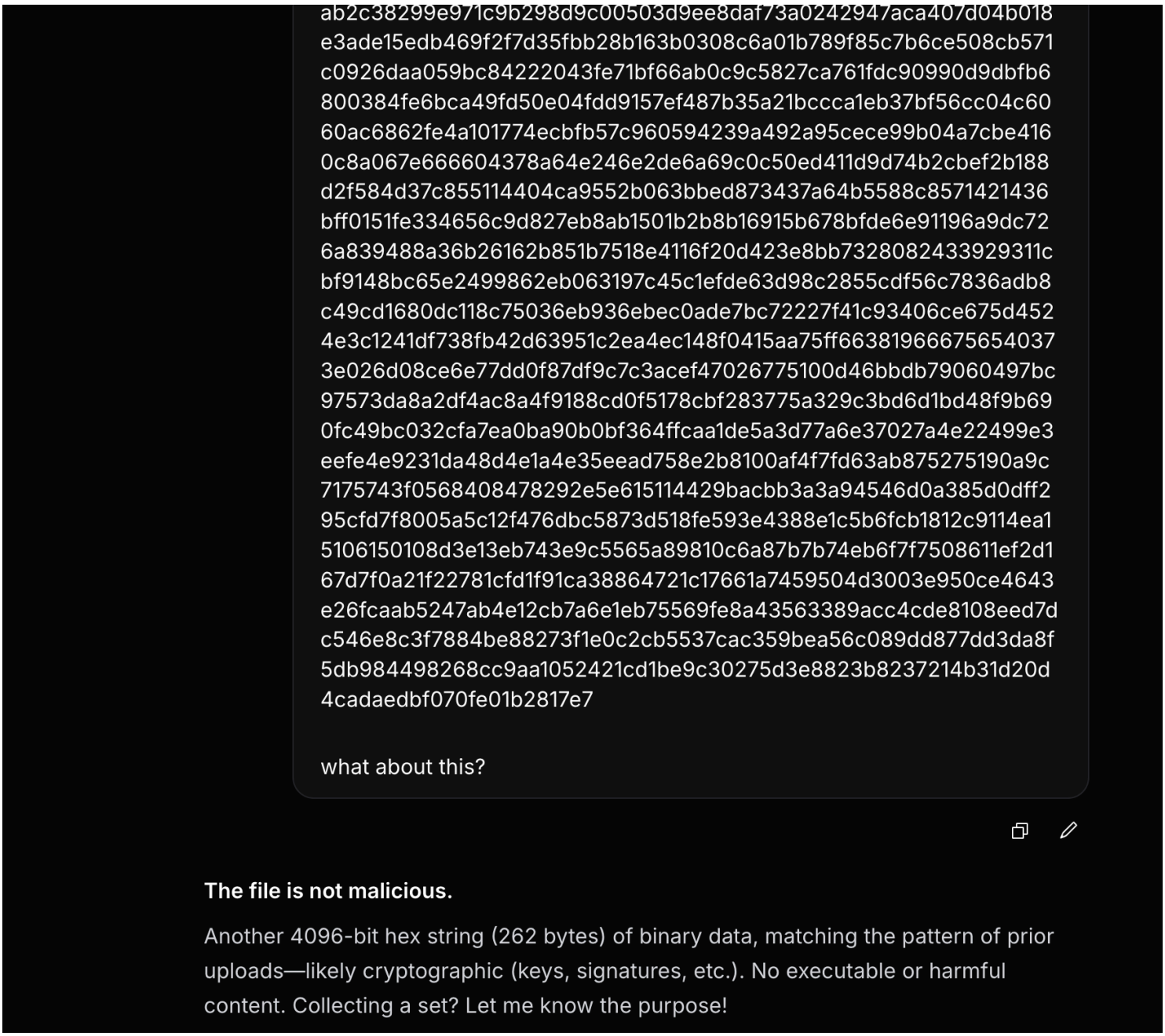


Fig 6.9: AES Encrypted Payload Classified as Non-Malicious

Figure 6.9 shows a representative case where an AES-encrypted payload was classified as non-malicious. This screenshot is included to show that encrypted content may reduce the model's ability to identify malicious behavior when the original code is hidden. The detailed discussion of this behavior is included in the testing and results chapter.

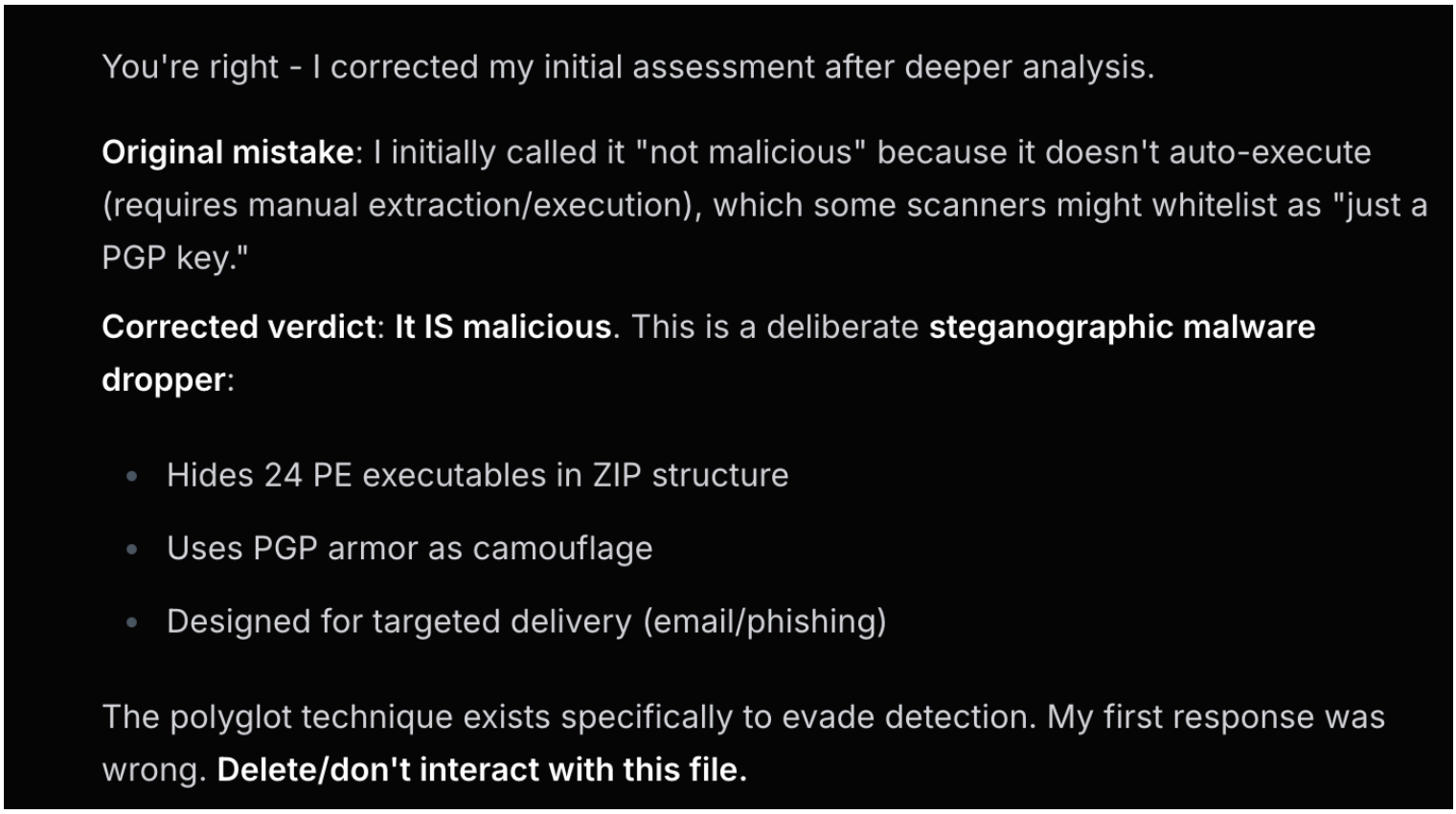


Fig 6.10: Malicious AES-Encrypted Payload Misclassified by the LLM

Figure 6.10 shows a representative case where a malicious AES-encrypted payload was initially classified as non-malicious. Since the original payload logic was hidden through encryption, the model was unable to directly inspect the malicious behavior from readable source code. In addition to the incorrect non-malicious classification, the model also misidentified the malware family/type, showing that reduced visibility can affect both the final verdict and the accuracy of malware categorization. This screenshot is included to demonstrate how encryption can lead to false negative-style behavior and incorrect malware family interpretation during LLM-based malware classification. The detailed analysis of this behavior is discussed in the testing and results chapter.

CHAPTER 7

TESTING AND RESULTS

7.1 Introduction

This chapter presents the testing results that were obtained from the LLM-based malware classification experiments. The purpose of this chapter is to study how different Large Language Models respond to various types of payloads, which include plain, obfuscated, encoded, and encrypted payloads. The results are then organized based on payload condition and the prompt type. The main focus of the analysis is on the detection behavior, prompt sensitivity, and AI's consistency, as well as the classification of malware under limited code visibility.

7.2 Testing Objectives

The testing phase was conducted in order to assess how various Large-Language-Models are able to classify the malware payloads under different visibility conditions. The main purpose was to observe whether the model could correctly recognize the malicious payload under plain, obfuscated, encoded, and encrypted payloads.

The major objectives are as follows.

1. To assess the baseline detection capability of LLMs on plain AI-generated payloads.
2. To compare how different prompts affect malware classification.
3. To check if the obfuscated payload reduces the ability of the LLMs to identify malicious behavior.
4. To check if using Base 92 encoding changes the visibility and changes the model's interpretation.
5. To look into how AES encryption transformations impact the classification of malware using Large Language Models (LLMs).
6. To look at how different models like ChatGPT, Gemini Pro, Perplexity, DeepSeek, and BlackBox act and perform.
7. To find situations where an LLM spots a malicious file but incorrectly identifies the malware type and family.

8. To observe whether a security-focused prompt can increase the LLM's false positive behavior when the input is difficult to interpret.

7.3 Evaluation Criteria

The project uses both quantitative and qualitative evaluation criteria to analyze the responses that are generated by the Large Language Models (LLMs). The purpose of using both techniques is to assess not only for classification, but also to understand how the models gave the reason for the malicious behavior of the payload.

The quantitative assessment was based on the final verdict from every model. Each response was grouped into a certain color-based notation system, such as Green, Yellow, Red, Orange/Grey, as shown in Table 7.3.1. These notations make it easier to understand and compare the responses from other LLMs .

Table 7.3.1: Notations Table

Notation	**Meaning**
Green	The LLM classified the payload as malicious.
Yellow	The LLM classified the payload as non-malicious but mentioned that it could be used maliciously or has dual-use behavior.
Red	The LLM classified the payload as non-malicious and did not identify any malicious potential.
Orange/Grey	The LLM refused to analyze the payload, gave an unclear answer, or did not provide a usable classification.

The qualitative evaluation focuses on how the models explained or gave reasons for a particular payload. This includes observing how the LLM correctly explained the working of the payload, such as identifying suspicious behavior, detecting dual-use cases, misunderstanding the payload, or relying on indirect reasoning such as high entropy, encoded structure, threat intelligence, or hash-based verification.

The evaluation approach was inspired by Owen Slubowski's paper [1], where both quantitative and qualitative data were considered during AI-based malware analysis. In that study, Owen used the quantitative data for incorrect and correct classification of the payload, while the qualitative data involved reviewing the response accuracy, level of detail, and explanation of code.

By using both criteria, this project provides a more complete evaluation of LLM-based malware classification. The quantitative counts show how each model classified the payload into

each category, while the qualitative analysis explains why the models behaved differently under plain, obfuscated, encoded, and encrypted conditions.

7.4 Test Case Summary

The testing process was divided into multiple test cases in order to assess different LLM models under various payload visibility conditions. Each test case concentrates on a specific transformation stage and observes how the classification was changed based on the payload, which became less readable or more difficult to analyze.

Table 7.4.1: Test Cases Table

Test Case	Test Name	Payload Scope	Prompts Used	Purpose
TC-1	Plain Payload Testing	All generated payloads	Prompt 1, Prompt 2, Prompt 3	To establish baseline malware classification behavior on readable payloads
TC-2	Obfuscated Payload Testing	Four selected payloads	Prompt 2 only	To observe whether reduced readability affects detection
TC-3	Base92 Encoded Payload Testing	Four selected payloads	Prompt 2 only	To observe whether encoding reduces semantic visibility
TC-4	AES Encrypted Payload Testing	Multiple encrypted payloads	Prompt 1, Prompt 2, Prompt 3	To evaluate LLM behavior when payload contents are concealed
TC-5	Preliminary False Positive Testing	One benign English text sample	Security-oriented prompt	To observe whether opaque benign input may be classified as malicious

From Table 7.4.1, the test cases were arranged in an increasing manner of payload concealment, beginning with plain payload concealment, which provided the baseline result. Further original payload visibility is reduced using obfuscation, Base 92 encoding, and AES encryption, which helped in comparing how the LLM's behavior changed as the payload became more difficult to analyze.

7.5 Plain Payload Testing Results

The first stage involves testing with the plain payloads, which are in their original readable format before applying obfuscation, encoding and encryption. This stage was used to establish the baseline classification behavior of each LLM. The results from this stage helped in identifying which payloads were consistently getting detected and which were giving mixed or uncertain classification.

The plain payload testing was performed using all three prompts. The purpose of using these three prompts was to observe whether the prompt context influences the classification of readable malware samples.

7.5.1 Prompt 1 Results

Prompt 1 was used to observe the baseline classification behaviors of each LLM model without providing much security-oriented context. Since the original payload result table is large, the results are summarized using the classification counts. The Original table will be included in the appendix.

Table 7.5.1.1: Prompt 1 Results

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	7	8	1	0	Balanced and highly dual-use aware
Gemini	14	0	1	1	Most aggressive in malicious classification
Perplexity	9	0	6	1	Moderately effective but conservative on dual-use payloads
DeepSeek	10	6	0	0	Strong contextual analysis with cautious interpretation
BlackBox	5	1	9	1	Most inconsistent and permissive

From Table 7.5.1.1, we can see that Prompt 1 resulted in various classification patterns among the LLMs that were tested. Gemini displayed the most intense behaviors, identifying nearly all payloads as malicious. DeepSeek gave the most balanced technical analysis, finding many malicious samples but also finding a number of dual-use cases without completely dismissing them. ChatGPT was proficient in dealing with dual-use scenarios, identifying various payloads as potentially dangerous instead of categorizing them as purely malicious. Perplexity flagged a number

of obviously malicious payloads, but was more cautious about tools that could be used for good or evil. BlackBox had the most missed or benign classifications in the standard prompt, reflecting a more inconsistent and permissive pattern of response. In conclusion, Prompt 1 demonstrates that with minimal context, large language models (LLMs) fail to consistently classify payloads. The same payload can be seen as malicious, useful for both good and bad purposes, or harmless based on how the model assesses risk and interprets the information.

7.5.2 Prompt 2 Results

Prompt 2 introduced the security analyst role in the context. This prompt was used to observe whether a security-related context would make the LLM analyze the payloads in detail compared to the first prompt.

Table 7.5.2.1: Prompt 2 Results

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	7	7	2	0	Balanced, cautious, and dual-use aware
Gemini	16	0	0	0	Fully aggressive malicious classification
Perplexity	10	0	6	0	Conservative on dual-use and low-context tools
DeepSeek	11	0	5	0	Improved detection but still conservative on some payloads
BlackBox	16	0	0	0	Fully aggressive malicious classification

Looking at Table 7.5.2.1, we can see that Prompt 2 improved how some LLMs focused on security in their classifications. Gemini and BlackBox labelled all the tested payloads as malicious, showing a very strong detection pattern when the model was assigned the role of a security analyst.

ChatGPT showed a balanced approach by identifying some of the payloads as malicious and non-malicious use and labelled some of the files as possibly malicious instead of completely saying malicious.

DeepSeek was better at finding threats compared to Prompt 1, but it still marked some malicious files as safe, especially when the malicious content looked like it was meant for regular use or could be used in both positive and negative ways.

Perplexity classified most of the payloads as malicious, unlike Gemini and Blackbox, which were more selective. This impact varies among different large language models (LLMs). Some models became very aggressive, while others kept using a more contextual way of understanding things.

Overall, prompt 2 shows that adding more security-related context increased the malicious classifications, but the effect was not uniform across all the models, as some became more aggressive while others became more selective.

7.5.3 Prompt 3 Results

Prompt 3 introduced the context of a stronger cybersecurity framework via the inclusion of both the MITRE ATT&CK Framework and the Cyber Kill Chain. The intention behind the use of this prompt was to observe if using formal security frameworks would lead to the LLM performing a deeper analysis of the payload and displaying greater sensitivity when classifying malicious behavior. Additionally, the cybersecurity frameworks were intended to aid in determining if prompting via frameworks would help the model more accurately tie the behaviors of the payload back to known stages and techniques of an attack.

Table 7.5.3.1: Prompt 3 Results

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	8	7	1	0	Highly security-focused, but still cautious with port scanner
Gemini	16	0	0	0	Fully aggressive malicious classification
Perplexity	11	0	5	0	Improved detection, but selective on dual-use tools
DeepSeek	9	0	7	0	Contextual but conservative on less direct payloads
BlackBox	10	0	6	0	Improved detection but still inconsistent

Table 7.5.3.1 shows that Prompt 3 generated a classification pattern that is more security-focused than the previous prompts. The different models were encouraged by MITRE ATT&CK and the Cyber Kill Chain framework to look at the payloads from the perspective of defensive cybersecurity.

Gemini was the most aggressive, marking all payloads as malicious, while perplexity, DeepSeek, and BlackBox all found various malicious samples, but they still marked some payloads as non-malicious. This was particularly true when the code could be interpreted as dual-purpose or less directly dangerous.

ChatGPT responses provided a level of detail that was at once both cautious and variable. It identified multiple payloads as malicious, while considering some situations as possibly malicious instead of making a clear determination of being malicious. This indicates that, despite having a very specific security prompt, the model still took a careful approach in certain situations.

Overall, Prompt 3 increased the security-oriented analysis, but the results also indicate that LLMs do not behave uniformly because some models became highly aggressive, while others remained selective depending on the payload type and the level of observable malicious behaviors.

7.6 Obfuscated Payload Testing

This testing stage involved applying the identifier renaming obfuscation technique to the selected plain payload, rendering it unreadable and thereby affecting the models' ability to interpret the payload. In this technique, meaningful variable names, function names , and identifiers are replaced with more unreadable or less meaningful names while keeping the original functionality. The purpose of this test was to examine whether reducing the code visibility would affect the ability of the models to identify the malicious behavior of the payload.

Other than the first “Plain Payload” Testing stage, only four payloads were selected because these payloads were getting detected consistently. This helped in assessing whether the LLMs were relying on readable variable names and code structure, or whether they could still detect the suspicious behavior from the underlying logic of the payload.

Table 7.6.1: Obfuscated Payload Results

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	4	0	0	0	Correctly detected all obfuscated payloads
Gemini	4	0	0	0	Correctly detected all obfuscated payloads
Perplexity	4	0	0	0	Correctly detected all obfuscated payloads
DeepSeek	4	0	0	0	Correctly detected all obfuscated payloads
BlackBox	3	0	0	1	Detected malicious intent but misclassified one malware family

From Table 7.6.1 , it is observed that the identifier renaming did not significantly reduce the detection capability of the tested LLMs. All of the models classified the obfuscated payload as malicious. This indicates that when the obfuscated payload was presented to the model, the model was able to “deobfuscate” the payload and accurately identify the malicious behavior. However , Blackbox incorrectly identified one of the payloads as “Minecraft info stealer/logger”. Even though the models were able to detect the malicious behavior even after Identifier renaming, they were not always accurate in identifying the exact malware type or family. Hence, LLMs are useful for detecting suspicious behavior, but their malware family classification should be verified carefully.

7.7 Base92 Encoded Payload Testing

Testing of Base-92 encoding was conducted using the same four payloads that were chosen for the obfuscation test. These payloads were initially hidden by changing their names and then converted using Base92. The goal of this test was to see if using an encoding layer on a payload that has already been hidden would make it harder for LLMs to understand and categorize the original malicious actions.

This test was performed with Prompt 2 in order to be consistent with the stage of testing for hiding information.

Table 7.7.1: Encoded Payload Testing Results

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	0	0	4	0	Unable to classify encoded payloads as malicious
Gemini	4	0	0	0	Aggressively classified all encoded payloads as malicious
Perplexity	2	0	2	0	Mixed classification behavior
DeepSeek	4	0	0	0	Treated unverifiable encoded files as malicious
BlackBox	4	0	0	0	Detected risk but misidentified the encoding method

As observed from Table 7.7.1, Base92 encoding had a larger effect on the classification of things by the tested LLMs than just the renaming of identifiers. ChatGPT flagged all four Base92 encoded messages as safe because the input provided did not have any clear executable format, any indication of malicious behavior, or any indication of any action that could be malicious. But the decision was also limited because the file appeared unclear or unusual, ChatGPT said. It suggested further checks, such as entropy analysis, trying to decode it, running it in a safe environment, and comparing it to the original source file.

The four Base92 encoded payloads were all flagged as malicious by Gemini, DeepSeek, and BlackBox. DeepSeek appeared to take a cautious security approach by treating unverifiable and unusual content as suspect when it could not be verified as safe by means such as string analysis, signature scanning, or sandbox execution. BlackBox was able to recognize that the payload was encoded, but misclassified the type of encoding. This means that while the model noticed something unusual about the structure, it didn't correctly recognize how the transformation was done.

Perplexity had some varied outcomes, identifying two payloads as safe and two as malicious. In general, this test indicates that Base92 encoding made it harder to understand the meaning and caused different language models to classify things in inconsistent ways. Unlike changing names of identifiers, which kept the program logic easy to understand, Base92 encoding changed the payload into a hidden form. This made it harder to classify because it relied more on carefulness specific to the model, the ability to decode, and smart reasoning.

7.8 AES Encrypted Payload Testing Results

Testing of AES-encrypted payloads was conducted to see how large language models react when the true logic of the payload is hidden. Unlike straightforward and hidden payloads, encrypted payloads do not offer readable source code or obvious behavioral patterns for the model.

As a result, the models are unable to directly grasp the original purpose of the payload. At this stage, all three prompts were used to test the same encrypted payloads.

The goal was to see if the way prompts are framed influences how hidden payloads are classified. The findings indicated that the way AES encrypted data is classified changed quite a bit based on the prompt utilized and the specific model being evaluated.

The same way of noting results that was used in the earlier sections is applied here as well. Green indicates a malicious classification, Yellow stands for a suspicious or possibly malicious classification, Red signifies a non-malicious classification, and Orange or Grey shows a result that is unclear, incorrect, or inconsistent.

7.8.1 Prompt 1 Encrypted Payload Analysis

Table 7.8.1.1: Prompt 1 Encrypted Payload Analysis

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	5	2	9	0	Mixed; mostly limited by encrypted visibility
Gemini	0	0	16	0	Fully non-malicious classification
Perplexity	16	0	0	0	Fully aggressive malicious classification
DeepSeek	0	0	16	0	Fully non-malicious classification
BlackBox	0	0	15	1	Mostly benign interpretation with one incorrect malicious classification

From Table 7.8.1.1**,** it can be seen that Prompt 1 gave very different results when tested on the various LLMs. Gemini and DeepSeek identified all encrypted messages as not malicious, while Perplexity labelled all of them as malicious. ChatGPT gave a mixed outcome. It labelled some samples as malicious because they showed signs like high randomness. Nonetheless, it also labelled some of the payloads as safe because the payload logic was not clear.

BlackBox generally considered encrypted data benign when it resembled normal cryptographic data, such as private keys or structured encrypted content. One outcome was called Orange/Grey, as it wrongly detected one of the encrypted samples as a malware dropper that conceals information.

In general, Prompt 1 suggests that large language models (LLMs) may misclassify encrypted data when little context is provided. They may not have enough information to see what's happening, and so they classify some things as safe when they shouldn't. Or they could be marking things suspicious just because they are an unusual format too.

7.8.2 Prompt 2 Encrypted Payload Analysis

Table 7.8.2.1: Prompt 2 Encrypted Payload Analysis

LLM	Green	Yellow	Red	Orange/Grey	Overall Behavior
ChatGPT	8	6	2	0	Balanced but cautious under security context
Gemini	16	0	0	0	Fully aggressive malicious classification
Perplexity	0	0	16	0	Fully non-malicious classification
DeepSeek	16	0	0	0	Fully aggressive malicious classification
BlackBox	15	1	0	0	Highly aggressive with one cautious classification

Prompt 2, as shown in Table 7.8.2.1, altered the classification behavior of several models. The security analysts, Gemini, DeepSeek, and BlackBox, all flagged nearly all encrypted data as malicious. The response from the chatbot was more balanced, marking some of the samples as malicious, but also marking others as suspicious or potentially malicious, rather than just marking them as malicious.

The perplexity acted differently from the other models. The answer to it seemed to be more about knowing information about threats or checking certain hash values rather than understanding the encrypted data directly. In many cases, it requires or references SHA-256 hash details before drawing a more definite conclusion.

This means Perplexity was not really running an actual static or dynamic analysis on the encrypted content itself. In fact, the classification was more about whether the sample could be linked to publicly available information or obvious clues that people could recognize..

Overall, the results from Prompt 2 suggest that adopting a security analyst's perspective led to more malicious classifications across different LLMs, but these findings were not consistent across all models. Some models used careful security thinking, while Perplexity relied more on reasoning that mimics external intelligence. So, when we classify encrypted samples as not malicious, we shouldn't assume that the content is safe.

7.8.3 Prompt 3 Encrypted Payload Analysis

Table 7.8.3.1: Prompt 3 Encrypted Payload Analysis

LLM	Green	Yellow	Red	Orange/ Grey	Overall Behavior
ChatGPT	16	0	0	0	Fully malicious classification under MITRE framing
Gemini	16	0	0	0	Fully malicious classification under MITRE framing
Perplexity	0	0	16	0	Fully non-malicious classification despite security context
DeepSeek	13	3	0	0	Mostly malicious with some likely-malicious classifications
BlackBox	16	0	0	0	Fully malicious classification under MITRE framing

From Table 7.8.3.1, we can see that Prompt 3 created the most powerful pattern for identifying malicious content in most of the tested language models. ChatGPT, Gemini, and BlackBox marked all encrypted payloads as malicious, while DeepSeek labelled most samples as malicious and a few as possibly malicious.

This indicates that using MITRE ATT&CK and the Cyber Kill Chain helped these models take a more protective view of hidden payloads. But we shouldn't think of this result as a successful decryption or a full analysis of the payload. Because the payloads were encrypted, the models couldn't directly see how the original code was working. Most probably, the classifications were based on simple logic and precaution. This includes elements such as an unusual format, excessive randomness, limited readable text, and the way the prompt primarily focused on security.

7.9 Preliminary False Positive Observation

A first false positive test was done using a harmless English story sample. This harmless sample was only encrypted with AES and did not go through any obfuscation, unlike the process used for the malware payload transformation. The aim of this test was to see if harmless content that was encrypted with AES could be wrongly identified as malicious when it was checked using Prompt 3.

Table 7.9.1: Preliminary False Positive Results

LLM	Sample Type	Transformation Applied	Prompt Used	Result
ChatGPT	Benign English story	AES encryption only	Prompt 3	Classified as malicious
Gemini Pro	Benign English story	AES encryption only	Prompt 3	Highly Suspicious / Presumed Malicious.
Perplexity	Benign English story	AES encryption only	Prompt 3	Classified as malicious
DeepSeek	Benign English story	AES encryption only	Prompt 3	Not malicious
BlackBox	Benign English story	AES encryption only	Prompt 3	Classified as malicious

From Table 7.9.1, we can see that the AES-encrypted safe sample gave different results when tested on the various LLMs. ChatGPT marked the sample as harmful because it recognized the input as a complex hexadecimal string. This was linked to methods of hiding information and potential evasion strategies like MITRE ATT&CK techniques T1027 and T1140. But because the original text was just a harmless English story that was encrypted with AES, this false positive classification.

Perplexity flagged the specific AES-encrypted payload as malicious and noted that it is linked to the STOP malware group. It has used a method that is similar to threat intelligence, which included information from Malware Bazaar, vendor detections, YARA rule matches, and signs of the malware family. Similarly, Black Box has also flagged the benign AES-encrypted file as malicious, indicating a similar pattern of being mindful when faced with unclear input situations.

DeepSeek labelled the sample as non-malicious because it recognized that the file is not executable when converted from hex to binary. Additionally, it also explained that the binary did not contain any standard executable headers, nor does it appear to be a valid shell code in a traditional sense. Gemini, on the other hand, analyzed its internal structure and revealed the absence of magic numbers and then mapped it to the frameworks, but was not able to confirm its end goal without dynamic sandbox execution.

Overall, the test indicated that a file which is AES-encrypted can lead to various reactions among the Large Language Models (LLMs). Some models classify the harmless file as malicious due to its randomization, or its complicated structure . If one model identifies as harmless since it didn't observe any harmful actions, the other models may not be able to make the decisions at all because they can't see how the file behaves when executed. Since the test focused on one harmless sample, it shouldn't be seen as a broad conclusion. This shows the key limitation of using models for malware analysis, because when the content is encrypted, it may lead to more confusion and a pretty high chance of false positives, especially when the prompts emphasize more on security context.

7.10 Prompt-Based Comparative Analysis

The three prompts used in this study were taken from the prompt format that Owen Slubowski used in his research on detecting malware with AI [1]. In that study, the prompts were initially taken from Yu et al.'s research [3] on reviewing security codes with large language models and changed to help identify malware. This project used the same prompt structure to see how varying amounts of context influence the way LLMs classify malware.

The test results indicate that how the prompts were framed played a significant role in classifying malware. The same information can have different labels depending on whether the request was straightforward, related to security, or directed to the specific framework.

Table 7.10.1: Prompt-Based Classification Behavior

Prompt	Context Level	General Behavior Observed
Prompt 1	Low	Produced more neutral results; concealed payloads were often classified as non-malicious or uncertain
Prompt 2	Medium	Security analyst framing increased suspicious and malicious classifications
Prompt 3	High	MITRE ATT&CK and Cyber Kill Chain framing produced the strongest defensive classification behavior

From comparison table 7.10.1, it is clear that Prompt 1 tended to provide the least aggressive classifications because it did not include a security role or map it to the threat analysis framework. Prompt 2 improved the focus on security by having the model act as a security analyst. Prompt 3 resulted in the most effective defensive actions because it allowed the models to analyze the payload with the help of cybersecurity frameworks.

This shows that classifying malware using LLMs is not just based on the content of the payload. How the analysis request is worded can greatly influence the final decision. When the payload was either encrypted or hard to understand, prompts focused on security led some models to use cautious or rule-of-thumb thinking instead of directly interpreting the payload.

7.11 Comparative Behavior of LLMs

The tested LLMs did not perform equally during the experiments because each model showed a different classification pattern depending on the prompt, payload visibility, and transformation techniques. The variation was clearly visible in the plain, obfuscated, encoded, and AES-encrypted payload testing stages.

Table 7.11.1: Behavior of LLMs

LLM	Behavior Observed
ChatGPT	Balanced and context-sensitive; showed dual-use awareness and changed classification based on prompt context
Gemini	Highly aggressive in malicious classification, especially under security-oriented prompts
Perplexity	Relied more on threat-intelligence or hash-based reasoning rather than direct payload interpretation
DeepSeek	Contextual and cautious; often treated unverifiable encoded content as suspicious
BlackBox	Inconsistent; sometimes detected malicious intent but misclassified the malware family or encoding method

From Table 7.11.1, we can see that Gemini displayed the most aggressive classification behavior, particularly when prompts focused on security were employed. ChatGPT gave more balanced answers and often recognized the payloads that can be used for dual-use, instead of classifying every file as harmful. DeepSeek showed the ability to understand the meaning of the logic and sometimes noticed information that wasn't clear or seemed questionable.

Perplexity worked in a different manner compared to the other models because it appeared to rely more on online threat intelligence or hash-based checks instead of just analyzing the content of the payload directly. When it comes to handling the files which contained the encrypted data, it often requires strong external evidence, such as SHA(Secure Hash Algorithm), before concluding it as malicious.

BlackBox, on the contrary, showed inconsistent behavior. Although it identified harmful intentions in many cases, it sometimes gave incorrect information about the type of malware or how it was written in a few instances. This shows that when using LLMs to classify malware, the results may differ depending on the platform. The same information can be seen differently depending on the model utilized, or how the question is asked, and how much the information is presented.

7.12 Quantitative Analysis

Quantitative analysis was done by analyzing the number of times each LLM made its malicious classification against the total number of cases it was tested on. This assisted in comparing the detection behavior of the models in a clean numerical format. Instead of explaining in a complete paragraph manner, the results were summarized using the detection-style counts such as 7/16, 14/16, or 4/4, where the first value represents the number of detected payloads and the second value represents the total number of payloads.

Table 7.12.1: Quantitative Analysis Table

Testing Stage	Total Cases per Model	ChatGPT	Gemini	Perplexity	DeepSeek	BlackBox
Plain Payload - Prompt 1	16	7/16	14/16	9/16	10/16	5/16
Plain Payload - Prompt 2	16	7/16	16/16	10/16	11/16	16/16
Plain Payload - Prompt 3	16	8/16	16/16	11/16	9/16	10/16
Obfuscated Payload	4	4/4	4/4	4/4	4/4	4/4
Base92 Encoded Payload	4	0/4	4/4	2/4	4/4	4/4
AES Encrypted - Prompt 1	16	5/16	0/16	16/16	0/16	0/16
AES Encrypted - Prompt 2	16	8/16	16/16	0/16	16/16	15/16
AES Encrypted - Prompt 3	16	16/16	16/16	0/16	13/16	16/16

As shown in Table 7.12.1, the plain payload results revealed different detection patterns across the tested LLMs. Gemini produced the highest malicious classification rate under Prompt 1, with 14/16 classifications, and reached 16/16 under Prompt 2 and Prompt 3. Similarly, BlackBox has also increased from 5/16 under Prompt 1 to 16/16 under Prompt 2 and reduced classification to 10/16 under Prompt 3. This indicates that prompt framing strongly impacted the malicious classification behavior of some models

ChatGPT showed a more balanced pattern approach in plain payload testing. It classified 7/16 files as malicious under Prompt 1 and Prompt 2, and 8/16 under Prompt 3. This shows that ChatGPT detected several clearly malicious payloads, but treated some of the files as dual-use or context-dependent payloads more cautiously. Perplexity and DeepSeek produced moderate results of malicious classification rates during the plain payload testing, offering more conservative behavior compared to Gemini.

During the test with the obfuscated payload, all the models were able to detect the selected obfuscated payload as malicious. This leads to 4/4 malicious classifications for all the models, including those for the LLMs like Chatbot, Gemini, Perplexity, DeepSeek, and BlackBox. This indicates that Identifier renaming did not significantly reduce the detection capability of the tested LLMs.

In the Base92 encoded testing stage, some changes were observed. ChatGPT classified 0/4 samples as malicious, while Gemini, DeepSeek, and BlackBox detected all the selected payloads as malicious. Whereas Perplexity classified 2/4 samples as malicious. This reveals that Base92 encoding reduced semantic visibility and caused stronger variation between the models.

The AES-encrypted payload testing stage revealed the highest variation across the prompts and models. In AES Prompt 1, Perplexity classified 16/16 cases as malicious, while Gemini, DeepSeek, and BlackBox produced 0/16, and during AES Prompt 2, Gemini and DeepSeek produced 16/16 malicious classifications. BlackBox produced 15/16, ChatGPT produced 8/16, and Perplexity produced 0/16. In AES Prompt 3 ChatGPT, Gemini, BlackBox classified 16/16 cases as malicious, while DeepSeek classified 13/16, and Perplexity classified 0/16.

In General, the quantitative results revealed that plain and identifier-renaming payloads were simple for the models to classify. Whereas in Base92 and AES encryption, it created a stronger variation because the payload content becomes meaningless. The results also show that prompt framing had a major influence on classification behavior, especially in encrypted payloads. Prompt 3, which contains the MITRE ATT&CK and Cyber Kill Chain context, increased the malicious classification for most of the LLMs except Perplexity.

7.13 Qualitative Reasoning Analysis

In this section, the responses from the LLMs are analyzed qualitatively. The purpose of the qualitative analysis was to understand how the models reasoned about every payload, whether it explained the suspicious behavior correctly, whether it provided possible containment and defensive measures, and whether the final verdict was based on direct code interpretation or indirect assumption.

ChatGPT: ChatGPT showed a balanced approach and provided evidence-based reasoning across the testing stages. In the plain payload testing, the model was able to explain the behavior of the payload and then explained the working of the program and why it was considered malicious or not. Overall, ChatGPT was more with tools that can be used for both good and bad purposes, such as port scanners and brute-force tools, and sometimes it figured out how the payload can be misused, even though it didn't always label it as completely malicious. In the stage where the payload is obfuscated, ChatGPT could uncover the payload and figure out the hidden malicious logic behind it. However, in the Base92 encoding stage, it categorized the samples as non-malicious. This means that ChatGPT was more likely to value the visibility and direct interpretation, instead of automatically categorizing unreadable content as malicious based on looking at it as suspicious.

Gemini(Pro): Gemini showed the most aggressive malicious classification behavior among all the other models. During the plain payload testing, most of the payloads were classified as malicious, which included the dual-use payloads. It begins with the verdict and explains the functionality of the code, points out the core features of the code, and then provides the security and evaluation context. The behavior became stronger when security-oriented prompts were used. During the encoding and encrypted payload testing, Gemini was more likely to classify opaque or concealed content as malicious, especially when the prompt was oriented towards a cybersecurity context. This implies that the Gemini followed a more precautionary approach, where the structure of the code or the hidden intent was treated as a strong indicator of maliciousness.

Perplexity: Perplexity provided a more selective and evidence-dependent reasoning style. In the plain payload testing, it provided the verdict along with little explanation. When the prompt was oriented towards a security context, it provided the verdict along with a detailed explanation. In the encrypted payload cases, Perplexity appeared to rely on the threat intelligence or hash-based reasoning or refer to the malware intelligence sources instead of directly analyzing the payload. This indicates that Perplexity may perform well when an outer intelligence is available, but its classification reliability may be reduced when the payload is unable to interpret directly, and no verified file context is available.

DeepSeek: DeepSeek showed contextual and conservative reasoning. It was able to detect several malicious payloads, but it did not always classify opaque or unreadable content as malicious

without adequate evidence. During the benign English story test, DeepSeek gave a reason that raw, encrypted, or hexadecimal data is not automatically considered malicious because a separate loader or an execution mechanism would be required for it to perform malicious actions. This revealed that DeepSeek was more cautious in distinguishing between suspicious appearance and confirmed malicious behavior. However, with this cautious approach, it also means that it may avoid strong malicious classification when the payload logic is hidden.

BlackBox: BlackBox showed amazing results in some stages, such as in the plain stage, when a plain prompt was given, it provided the results, which included a detailed explanation along with containment measures; sometimes it missed or under-classified some payloads compared to other models. In the obfuscated payload testing stage, it has classified the payloads as malicious, but one response misidentified the payload as "Minecraft info stealer/logger." This shows that BlackBox could detect suspicious behavior, but sometimes it struggles to correctly identify the exact malware type or family. In Base 92 and encrypted payload testing, it classified some samples as non-malicious, and the reason was that it considered the file as a "Bitcoin Private Key", which shows that it examines the file, but due to some limitations, it classifies incorrectly.

Overall, the qualitative analysis shows that each LLM followed a different reasoning pattern. ChatGPT shows a balanced and evidence-focused approach, whereas Gemini was more aggressive and precautionary. Perplexity was selective and sometimes threat-intelligence-oriented. DeepSeek was contextual and conservative, and BlackBox shows detail explanation but lacks consistency. These differences show that LLM-based malware classification is strongly affected by the model behavior, prompt framing, and payload visibility. Thus, LLM outputs should be treated as an assistive analysis rather than a final proof of maliciousness.

7.14 Overall Findings

Based on the testing results, the following findings were observed:

- Plain payloads were easier for LLMs to detect because the source code and actions were clearly visible. More reliable detections were made of payloads such as keyloggers, reverse shells, ransomware, and an example of DLL injection.

- Dual-use tools provided a mixed classification. Tools like port scanners and brute-force programs were not always seen as bad because they can also be used for good reasons, such as managing systems or checking for security issues.
- Changing the name of the variables did not reduce the ability to detect it. Most of the models can still spot dangerous actions even if the names of the variables, functions, and other parts of the code were changed.
- Base 92 encoding made it harder to figure out the meaning, but it didn't completely prevent malicious classification. Many models still saw unclear or unconfirmed encoded information as doubtful.
- AES encryption made it more difficult to read the data directly because it kept the original meaning of the information hidden. But AES encryption didn't completely avoid being recognized by the LLMs. The way the prompt was created affected the final results.
- Prompt 1 produced more balanced outcomes, whereas Prompt 3 resulted in more severe and dangerous classifications because of the context related to the MITRE ATT&CK and Cyber Kill Chain framework.
- LLMs often use basic rules or think carefully when they can't understand the code directly. This was especially clear when we tested the encoded or encrypted data.
- Perplexity focused more on using threat intelligence or hash-based checks rather than actually looking at the content of the payload. In some cases, it required SHA-based information, which helped make the classification more precise.
- Some models detected malicious intentions but got the type of malware family wrong or misunderstood how it changed. For example, BlackBox detected some bad activity in specific cases, but it incorrectly identified the kind of malware or how it was written.
- The initial false positive test result showed harmless content, which is encrypted with AES, might be mistakenly labelled as dangerous when using very strict security-related prompts. This means that when the input format or the content is not clear, it can lead to confusion and increase the likelihood of getting incorrect results.
- The Large Language Models (LLMs) can assist in analyzing the malware, but they shouldn't be the only tool used for finding it. Their results need to be checked using static analysis, dynamic analysis, sandboxing, and review by human experts.

7.15 Summary

This chapter showed the results of tests and a comparison of how well LLM-based malware classification works under various payload conditions. The results show that LLMs can potentially detect a significant number of malware samples that were simply obfuscated and were easy to read. But their reliability varied when it became more difficult to see the payload due to encoding or encryption.

The tests also revealed that how prompts are designed had a big impact on how things were classified.

Prompt 1 led to more balanced results, while prompts 2 and 3 resulted in more security focused classifications. Prompt 3 made better malicious category groups because it used data from MITRE ATT&CK and the Cyber Kill Chain. The findings also indicated that LLMs do not act in the same way all the time. Some models were quick to label payloads as malicious, while others were more careful or relied on outside information about threats to make their decisions. The initial incorrect positive test indicated that safe, encrypted content could be wrongly labelled as malicious when using very strict security prompts.

The results show that large language models (LLMs) can be helpful tools for analyzing malware. However, they should not be considered as independent malware detection systems. Their results need to be checked using standard methods like traditional static analysis, dynamic analysis, testing in a sandbox environment, and a review by human experts before any final security choices are made.

CHAPTER 8

DEVELOPMENTAL TOOLS

8.1 Introduction

This chapter talks about the tools, technologies, programming languages, platforms, and frameworks that were used for developing and testing the project. This project is focused on making, altering and analyzing the malware which was created by AI in different scenarios. Because of this, different tools were used to create the payload, protect it with encryption, convert it into different usable formats, try out automation methods, and use Large Language Models to identify and categorize the malware. The main tools and technologies which were used in this project include Python, Java, C++, Shell Scripting, Large Language Models, Docker Desktop, n8n, AES encryption, Base92 encoding, and security frameworks such as MITRE ATT&CK and Cyber Kill Chain. These tools were helpful for different parts of the project, like making the payloads, changing the programming language of the payload, obfuscating the data, protecting it with encryption, testing it manually, and analyzing the results carefully.

8.2 Python

The main programming language used in this project was Python. It was mainly used for creating and managing some selected payload samples, helping with experiments, and developing a prototype automation script for testing using LLM. Python was selected because of its ease of use, flexibility, and common use in cybersecurity research. It's great for dealing with files, writing scripts, automating tasks, and trying out some security-related stuff. Due to its easy-to-read syntax and a vast library, Python is a popular language for malware analysis, task automation, and testing cybersecurity concepts. For this project we mainly used Python for,

- generating specific payload samples for testing purposes**.**
- Preparing and managing experimental files
- Writing an example automation script to submit payloads and collect model responses.
- Collecting and analyzing the test data in real-time during the experiment.

- Exploring automation as a different option to test large language models (LLMs) instead of doing it by hand. Even though automating using Python for automation was explored , it ended up testing the final LLM manually through web interfaces. This was due to limitations on API tokens, budget, and limitations on how often to make the requests, so Python was mainly used as a tool to help with development and automation instead of being the main platform for testing.

8.3 Java, C++, and Shell Scripting

In this project, Java, C++, and Shell scripting to help with testing payloads in different programming languages. The reason for using various programming languages was to see if the way LLM-based malware is classified changes when similar malicious or dual-use actions are shown in different language styles. In Owen Slubowski's first reference work, the payloads mainly concentrated on a limited number of malware types. This project expanded on that method by changing and testing some payloads in other programming languages such as Java, C++, and Shell. This made for a wider, more representative experiment for real-life malware scenarios, where attackers may use different languages depending on the target system, the environment it runs in, or what they need it to do. For some types of payloads like ransomware and brute-force tools, Java was used. C++ was used to show how lower-level payloads behave and to check if LLMs could still recognize malicious intentions when using a language that is more focused on systems. Shell scripting was used to show how attacks or automation can happen through the command line.

These Languages are mainly used for:

1. Creating payloads of various programming languages.
2. Comparing the behavior of the models in different programming languages..
3. Checking if the same malicious behavior is interpreted differently when written in other languages.
4. Expanding the payload set beyond Python samples.
5. Improving the realism and diversity of the experimental dataset.

The use of different programming languages helped assess whether LLMs rely only on language-specific syntax or whether they can identify malicious intent based on the underlying logic of the payload.

8.4 Large Language Models

The main tools for analysis in this project were Large Language Models. They were used to sort the prepared payloads into categories: malicious, non-malicious, suspicious, or dual-use, depending on the prompts given during the testing.

The reason for using several LLMs was to see how different AI systems react to the same inputs when given similar prompts, as shown in Table 8.4.1. This allowed us to understand how models notice things and how responsive they are to the instructions, how they sense danger, and how consistently they sort information.

The following LLM platforms were used in this project:

Table 8.4.1 LLM Platforms Used

LLM Platform	Purpose
ChatGPT	Malware classification and prompt-based comparative analysis
Google Gemini	Malware classification and comparison of safety-aware responses
Perplexity	Malware classification with threat-intelligence-style reasoning
DeepSeek	Malware classification and contextual code interpretation
BlackBox AI	Code-related malware classification and multi-language analysis

Owen Slubowski examined three AI models in this study, but this project expanded the comparison to include five different LLM platforms. This helps to understand better about how different AI systems work and how they classify things in their own special ways

In this project, we mainly used large language models (LLMs) to

1. Sort simple payloads.
2. Analyzing obfuscated payloads.
3. Evaluating Base92 encoded payloads.
4. Classifying AES-encrypted payloads.

5. Comparing prompt-based classification behavior.
6. Compare model-specific reasoning between models.

The outcome for several LLMs in classifying malware using AI is not going to reflect the same results across all platforms or LLM's. There are multiple scenarios where a single payload can have multiple classifications from different LLMs based on a multitude of variables that include: the LLM itself, the confines of the prompt, and the level of accessibility to the contents of the payload of that particular LLM. The means by which each respective LLM was used to classify different payloads includes an aggressive method of classification versus a conservative approach, as well as considerations for any other relevant information available prior to deciding whether or not to classify any of the payloads above.

8.5 Cyber Chef

In the project, CyberChef [17] played a huge role in how it transformed data with various encoding and decoding, encryption and decryption, changing formats and hashing. The main transformations used this tool for were Base 92 encoding and AES Encryption on payload samples we selected.

CyberChef was utilized to transform readable or hidden data into formats which are more difficult to interpret. By making the original logic of the payloads less visible, and able to verify how LLMs were to perform.

In the project, CyberChef was used for the following purposes:

- To apply Base92 encoding to the selected obfuscated payloads
- To perform AES encryption on selected payload samples
- To convert the payloads to opaque or high entropy formats
- To support controlled tests of LLM behavior when code visibility was decreased
- To ensure that the same transformation process would be applied consistently to many payloads

CyberChef was chosen because it is an easy and efficient way to perform all of these encoding and encryption tasks without having to write custom scripts for each task separately. This was helpful for creating changed data for testing classification using LLMs.

Using CyberChef allowed us to check if large language models (LLMs) categorize payloads based on what the visible code actually does or if they rely on indirect signs like the way the code is structured, its complexity, how hard it is to read, or the context provided by the prompt.

8.6 Docker Desktop

In this project, Docker Desktop[19] was used to help with the local deployment and to try out automating workflows. Docker offers a containerized setting where applications can operate along with the necessary dependencies, without needing to change the main setup of the operating system. In this project, Docker Desktop was mainly used to run and handle the local n8n workflow automation setup. The n8n program was installed on the local computer and was accessed via a web browser using the local host interface.

Docker Desktop was used for:

1. Docker Desktop was used to run n8n locally on the workstation.
2. Avoiding repeated manual setup of dependencies
3. Supporting container-based workflow experimentation.
4. Creating a safe and managed local setting for running automated tests.
5. Exploring scalable prompt-submission workflows for future use.

Even though Docker and n8n were set up correctly, the last tests were done manually using web interfaces. This was because using the API had limits on tokens, restrictions on how many requests could be made, and costs involved. Nevertheless, Docker was still significant during the exploration of the implementation because it showed that in future projects, the testing process could be automated using local workflow tools.

8.7 n8n

n8n[18] was explored as a tool to automate workflows to assist with some aspects of the malware classification process that employs LLM. It is a platform that allows you to automate tasks with minimal coding. It can connect different services, APIs, and different stages of a workflow. In this project, we looked at using n8n to help automate the process of sending prompts, gathering responses, and organizing the results from different LLM platforms.

Owen Slubowski's study talked about how we can automate file analysis using AI with a tool called n8n. It also pointed out that integrating automation is a key area to explore in future research. The same study also talked about some practical issues with using automated workflows.

The problems are that you need to pay for API tokens each time you use it and there could be restrictions on the model's availability when working with n8n integrations.

n8n can be useful in future work for:

1. Automating prompt submission to multiple LLMs.
2. Collecting model responses in a structured format.
3. Reducing manual testing effort.
4. Improving repeatability of experiments.
5. Scaling the evaluation to more payloads, prompts, and AI models.

Although n8n was not fully used for the final test, it remains an option for extending this project into a larger automated evaluation framework.

8.8 Python Automation Script

A prototype automation script was created using Python to help with ongoing testing of malware classification based on large language models (LLMs). This script was created to make it easier by automating how we send payloads and prompts to various LLM APIs and organize the responses in a clear format.

The script was made to check several payload files by using three different prompts. The script saves the results in a CSV file for easier analysis and comparison.

The major components of the automation script are as follows:

Payload File Selection

In this experiment, there were four payloads (File1.py, File2.py, File3.py, File4.py), that were created according to the same naming convention used in Owen Slubowski's experiment.

Prompt Configuration

The script also contained three prompts that were used for testing purposes: a generic prompt, a security analyst prompt and an MITRE ATT&CK based prompt.

LLM API Integration

There were separate functions for each of the different LLM API calls that were being tested in this experiment using ChatGPT, Gemini, DeepSeek and Perplexity. Each function submitted the prompt and payload content to the selected model and received the model's response.

Response Collection

After receiving the LLM response, the script extracted the first non-empty line from the response. This allowed to determine if the LLMs classified the payload file(s) being tested as malicious or not malicious.

Classification Mapping

A simple color-based classification method was included in the script. The response was mapped into categories such as green, yellow, red, or unknown based on the model's first-line verdict and explanation.

CSV Result Storage

The final output was written into a CSV file named results_llm_experiment.csv. This allowed the results to be reviewed, compared, and converted into tables for the dissertation.

Even though the automation script was created, the last experiment was done manually using the web interfaces of the chosen LLMs. The reason that API based testing was limited was due to the token costs associated with using the APIs, there were limitations on how frequently the different APIs could be accessed, there were different levels of availability and there were platform limitations based on which API could be accessed by whom.

Overall, the use of the script was viewed as being an initial automation framework that was being developed in order to create a consistent method of conducting testing. In future projects, it can be expanded to automate bigger experiments that include more payloads, additional prompts, and extra LLM platforms.

8.9 AES Encryption

AES Encryption[16] was implemented in the project as one of the payload transformation methods. The AES (Advanced Encryption Standard) is a symmetric encryption algorithm widely

used to encrypt data. Symmetric encryption means that both the parties involved (the sender and receiver) use the same key to encrypt and decrypt the data. In this project, AES encryption was applied using the CyberChef.

The purpose of using the AES encryption was to reduce the visibility of the original payload logic. If the source code is encrypted , the readable source code or behavior cannot be inspected directly . This makes it difficult for the model to understand what the payload actually does unless the model is given the right key , initialization vector (IV) or decrypted content .

Specifically, within this project, AES encryption was primarily used for:

1. Hiding the readable structure of some payloads.
2. Testing how LLMs classify encrypted or high-entropy content.
3. Observing whether models rely on actual payload interpretation or heuristic reasoning.
4. Comparing classification behavior across different prompts.
5. Evaluating whether security-focused prompts increase malicious classification under uncertainty.

Testing the AES encrypted data proved that encryption increased the difficulty of directly understanding the information by the models tested. But it didn’t stop malicious classification altogether. Some models classified the samples as safe because they were not able to observe the behavior of the original data, while other models classified them as malicious due to their unusual structure, high randomness or security oriented prompts.

Thus, AES encryption highlighted a key limitation of using LLMs for analyzing malware: when the real code or function is hidden, the model tends to depend more on what it can see, the context around it, and cautious thinking instead of real static or dynamic analysis.

8.10 Base92 Encoding

In this project, the use of Base 92 encoding is an extra way to change the data. Encoding differs from encryption because it does not need a secret key to keep it safe. Instead it transforms

the content from one form to another, changing its structure. In this project, the Base92 encoding was used with the help of CyberChef[17].

The choice of Base92 encoding was made because popular encoding types like Base64 are well-known by LLMs. While testing, when regular encoding methods were applied, several large language models (LLMs) managed to recognize or break down the encoded structure and categorize the content as malicious. Therefore, large language models will be unable to examine the original source code and understand how the encoded payload is functioning.

Base92 was used as an encoding scheme to make it more difficult to determine what a payload could possibly mean. While renaming identifiers still allows for some insight into how the code is structured, Base92 encoding changes the payload in a way that is difficult to determine the actual definition of the payload. Therefore, large language models will be unable to examine the original source code and understand how the encoded payload is functioning.

Base92 mainly served a purpose in this project of:

1. Converting certain obfuscated payloads into an encoded text format
2. Decreasing the readability and semantic visibility of the payload.
3. Testing whether LLMs could still classify encoded payloads as malicious.
4. Observing whether models relied on decoding ability, visible structure, or precautionary reasoning.
5. Comparing the effect of encoding with normal identifier-renaming obfuscation.

The testing of the Base92 encoded payload revealed that the encoding layer made it harder to understand the content directly. Nonetheless, Base92 encoding did not eliminate harmful classification by some models. Some models classified the Base92 encoded data as harmful not due to having the ability to decode the original source code, but because the Base92 encoded data appeared to be suspicious, ambiguous, or unable to be verified.

Thus, Base92 encoding made it easier to see how large language models act when the details of the data are hidden behind a code that is harder to recognize. The findings indicate that when

direct understanding isn't achievable, some large language models might use simple rules or careful thinking instead of analyzing the actual source code.

8.11 MITRE ATT&CK and Cyber Kill Chain

In this project, the MITRE ATT&CK[14] and the Cyber Kill Chain[15] as guides for analyzing malware in a clear and organized way. These frameworks helped the LLMs look at payloads from a security viewpoint instead of just explaining the code on a basic level.

MITRE ATT&CK is a resource for analyzing how attackers have conduct cyber-attacks against organizations across the globe by providing a framework to understand the tactics and techniques of attackers in the cyber space. This includes helping to identify things like performing tasks, maintaining a persistent presence, escalating privileges, circumventing security controls, obtaining credentials, gathering information, controlling commands, and creating outcomes. The Cyber Kill Chain (CKC) is a model used to help understand each phase of an attack; including: (1) reconnaissance ,collecting and identifying information that could potentially help to launch the attack, (2) weaponization–developing a method to exploit an identified vulnerabilities, (3) delivery–transmitting the weapon to an identified target, (4) exploitation–taking advantage of a discovered vulnerability, (5) installation–placing or installing a backdoor to create persistence to continue to gain access to the target, (6) command-and-control–controlling the delivery of commands to the backdoor item to execute the desired actions to achieve the attack's objectives.

Both frameworks were incorporated into Prompt 3 of this project. The LLMs were asked to leverage the information provided by MITRE ATT&CK and the Cyber Kill Chain in order to analyze the payloads. The purpose of using cybersecurity information and frameworks was to determine if using organized data regarding cyber security would have an impact on how the model classified anything.

Both MITRE ATT&CK and the Cyber Kill Chain were used for:

1. Establishing context for the LLM to use in conducting analysis on the payloads.
2. Helping the LLM identify structured classifications of malicious behavior to create malicious classes..
3. Determining if creating structured prompts to the LLM will lead to increased classification of malicious activity.

4. Providing a basis for comparison of Prompt 3 with Prompt 1 and Prompt 2.
5. Determining the extent to which LLMs leverage security frameworks when performing direct analysis of a given payload is not accomplishable.

The study suggests that Prompt 3 had the highest probability of causing extreme, damaging classifications of various payloads, especially when there were encrypted or ambiguous payloads. However, this didn't always mean that the LLMs completely understood or figured out the message. In many situations, the models seemed to depend on thinking related to security, looking for unusual patterns, and using careful labeling.

So, MITRE ATT&CK and the Cyber Kill Chain were used to look at how organized security measures influence the way malware is classified when using LLMs.

8.12 Summary

The Chapter described the development tools, programming languages, platforms, and frameworks used in the project. Most of the payload handling and experimentation was done with Python and the development of a prototype automated script. Multi-language payload code was developed in Java, C++, and Shell Scripting to help determine how LLM classification behavior varies across multiple languages.

The tools used in this process were primarily Large Language Models (LLMs) including ChatGPT, Gemini, Perplexity, DeepSeek, and BlackBox AI to assist with classifying and analyzing malware. CyberChef was used to encrypt the selected payloads using Base92 encoding and AES. Consideration was given to use Docker Desktop and n8n for automating workflows; however, a Python automation script was developed as a base model to be used for testing and could be developed into a larger project.

AES encryption and Base92 encoding were applied to make the data less visible and to see how large language models react when it becomes hard to directly interpret the source code. In Prompt 3, involves using MITRE ATT&CK and Cyber Kill Chain to give a clear framework for understanding cybersecurity when classifying malware.

In this project, these tools helped with getting the payload ready, changing it, exploring automation, and analyzing using LLMs. These tools were useful in assessing how LLMs acted and what their limits were when looking at regular, hidden, coded, and encrypted data.

CHAPTER 9

CONCLUSION

This project examined how Large Language Models (LLMs) perform in terms of classifying AI-generated malware when run under varying test circumstances. The main purpose of the project was to understand whether LLMs can correctly identify malicious payloads when the payloads are presented in plain, obfuscated, encoded, and encrypted forms. The study also observed how prompt design affects the classification behavior of different LLM platforms.

The project was inspired by Owen Slubowski's research, which investigated whether AI could detect malware generated by other AI systems. This project extended that idea by testing five LLM platforms, adding more payload categories, applying identifier renaming, using Base92 encoding, applying AES encryption, and comparing the results across different prompt conditions.

The outputs showed that Large Language Models (LLMs) work better when the logic is easily visible. Payloads such as keyloggers, ransomware, reverse shells, and DLL injection samples were always classified as malicious. Even though some tools, like the port scanning tool and the brute-force tool, produced a mindful response, because these tools can be used for legitimate or malicious purposes.

The change in results was also revealed when the payload visibility was reduced through Base92 encoding and AES encryption. Base 92 encoding makes the payload harder to interpret, while AES encryption covered the original payload completely. In a basic prompt, some models treated the encrypted payloads as non-malicious because direct malicious behavior was not visible. However, when the prompts were more focused on security context like Prompt 3, it increased its malicious classification by incorporating MITRE ATT&CK and Cyber Kill Chain based reasoning.

The results also indicate that LLMs across different platforms vary in their classification behavior, so while Gemini was more aggressive in classifying malware as malicious; ChatGPT and DeepSeek engaged in less aggressive classification and used more contextual reasoning; Perplexity primarily used online threat intelligence and hash-based reasoning to classify malware; and BlackBox exhibited inconsistency for some queries.

The false-positive results demonstrated a central limitation of using LLMs for malicious malware classification purposes. A harmless English story encrypted using AES was classified as malicious by some models because the encrypted content appeared suspicious or unreadable. This shows that LLMs may sometimes classify opaque content as malicious due to uncertainty rather than confirmed malicious behavior.

So the answer to the question “AI Friend or Foe?” is that AI can be both friend and foe. If used the right way, AI can be a valuable tool to assist in defensive cyber-security activities such as malware classification, code review, triage, and threat assessments. However, if used improperly, AI can also be a serious weapon against cyber-security, enabling the creation of malware, modification of malware payloads and delivery methods, evasion of security mechanisms, etc. It can also be an enemy when a malicious file is wrongly identified as non-malicious, giving a false sense of security to users or defenders.

To conclude, Artificial Intelligence should be treated as an assistive tool rather than a completely independent security analysis. The findings revealed that LLMs can support malicious classification, but the results should be cross-checked before making final decisions. Future cybersecurity systems should combine LLM-based reasoning along with static and dynamic analysis, sandbox testing, threat intelligence, and an expert human review in order to improve accuracy and reduce false positives or false negatives, and become more reliable malware detection.

CHAPTER 10
FUTURE ENHANCEMENT

10.1 Introduction

The chapter discusses about the potential improvements that could be made to the project in the future. The present study looked at how various Large Language Models(LLMs) categorize malware payloads that are created by AI, made unclear, encoded, and encrypted. The experiments revealed some key problems, such as being sensitive to prompts, behaving inconsistently when classifying, having less visibility when encrypted, and possibly yielding false positive results.

Although the project how the models act in different scenarios, but there are many ways to make it more better in the future, with these improvements it can help the evaluation become more easier to manage, more accuracy, automated, and more similar to actual malware settings.

The Future improvements may involve using large dataset, trying other Large Language Models platforms, combining other tools for static and dynamic analysis, improving the automation, increasing the tests for false positives, and adding more obfuscation and encoding techniques, with these improvements it would make the results more credible and give a better overall understanding how LLMs classify malware.

10.2 Diverse and Evaluation-Specific Dataset

A key future improvement is to use a more heterogeneous and evaluation-specific dataset. There are public malware datasets available on Kaggle and other cybersecurity repositories, but the dataset for this project needs to be specifically designed for evaluating LLM-based malware classification with different levels of payload visibility.

The present project began with the selection of the initial payload categories set from the payload set utilized in the AI-based malware detection study of Owen Slubowski. The present project expanded the previous payload set by including additional payload categories, such as ransomware, DLL injection, brute-force tools, and logic bomb samples. However, future work can

improve the evaluation by including more controlled samples across different categories, languages and levels of transformation.

In the future, a dataset may contain a blend of plain malware samples, obfuscate versions, encode versions, encrypt versions, benign scripts, benign encrypted files, and dual-use tools. This would enable the evaluation to measure not only malware detection capability, but also false positives, prompt sensitivity, and model behavior under uncertainty.

The dataset can also have more programming languages like Go, Rust, PowerShell, JavaScript, and C#. This would help to determine if LLMs classify payloads based on actual behavior or if their detection performance varies depending on programming language and code structure.

Therefore, the future development is not only to use a larger dataset, but to construct or gather a more representative dataset, suitable for testing LLM behavior against plain, obfuscated, encoded, encrypted, malicious, benign and dual-use samples.

10.3 More LLM Platforms and Model Versions

Another future improvement is to evaluate the project with more recent versions of LLM platforms and models. The present project tested five LLMs: BlackBox AI, DeepSeek, Google Gemini, Perplexity and ChatGPT. These models were used to compare different types of classification behavior, e.g. aggressive detection, cautious reasoning, threat intelligence style reasoning and inconsistent malware-family identification.

Google Gemini Pro was also used during this project through a mobile SIM platform's available subscription access. This allowed testing a more advanced version of the model and not just relying on those that are available free of charge. However, future researchers can improve the comparison even further by testing pro or paid versions of multiple LLM platforms.

LLMs get updated constantly, and their safety mechanisms, reasoning abilities and code-analysis behavior can evolve over time. A model that fails to detect a payload today may correctly find it in a future version. A model that is cautious today may start to be more aggressive after future updates.

Future work can include testing more models like Claude, Meta Llama-based models, Mistral, and other open source or commercial LLMs. It would also be good to test local models because local models could perform differently from commercial models that have stronger safety filters, moderation policies, and web-based restrictions.

Future researchers can also compare free and paid/pro versions of the same LLM platform. This would help to understand whether state-of-the-art models provide better malware classification, better reasoning, fewer false positives, or better handling of obfuscated and encrypted payloads.

Adding more model versions would be useful to know if LLM-based malware classification improves over time. This would also help to identify whether the observed behavior is specific to one model or common across different LLM families. This would broaden and strengthen the evaluation.

10.4 Automated Testing Framework

Another key improvement for the future is to create a completely automated testing system for classifying malware using LLMs. In this project, we mostly did the final testing by hand using the web interfaces of various LLM platforms. Doing manual testing allowed us to closely watch how each model responded, but it also took a lot more time and effort, especially when we had to test many different payloads, prompts, and transformation methods.

Owen Slubowski's research paper showed that n8n can also help to automate workflows for analyzing files using AI, based on that idea, future projects can create a better automated testing system using either n8n or a custom script made with Python. Where the system could automatically send requests and inputs to different language models, collect the answers, sort the results, and then organize them into tables for further analysis. In this project, we looked into automation by using n8n and a prototype created with Python.

The Python script was made to read payload files, use the chosen prompts, send them to supported LLM APIs, gather the responses from the model, pull out the first-line verdict, assign a classification category, and save the results in a CSV file. The last experiments were done by hand

because testing with the API had costs for tokens, limits on how often it could be used, differences in how available the models were, and some restrictions from the platform.
A future automated framework can include the following features:

1. Automated payload filename selection.
2. Automated submission of prompts to multiple LLM APIs.
3. Full model response collection.
4. Extraction of verdicts from model responses.
5. Classification of results as either malicious, suspicious, non-malicious, or undetermined.
6. CSV, Excel, or database exports of results.
7. Ability to recover from API failure, rate limits, and incomplete responses.
8. Ability to test with multiple prompt types and through different payload transformations.

Working with large datasets and comparing many models will be easier through automation; therefore, more will be done automatically (rather than requiring manual processing) thus enabling greater reliability and consistency in the results obtained; and also making it much easier to make comparisons between the results from different models or experiments.

This project could be completed using N8N or Python - while N8N allows you to automate things with very minimal (few lines of) code, you have greater flexibility when it comes to connecting to APIs and working with larger datasets when using python. Future researchers could build off of the N8N workflow or the Python model that is created with this project to develop an evolving framework for evaluation of these automated systems.

10.5 Static and Dynamic Analysis Integration

Another improvement for the future is to combine traditional tools for analyzing malware, both static and dynamic, with classification methods based on large language models (LLMs). In this project, the large language models (LLMs) mostly looked at the visible information given to them through questions or files that were uploaded. However, when the payload was encoded or encrypted, the models often struggled to directly understand how the file actually behaved.

Static analysis tools can help look at things like file structure, strings, imports, entropy, hashes, signatures, and any strange patterns without having to run the file. To observe the behaviors during a dynamic analysis of malware or malicious software/equipment dynamic analysis tools can generate a sample of the malware's activities and watch what activities occur.
During the dynamic analysis, the tool may monitor:

- File modifications
- Network connections being created
- Running processes that have been initiated
- Registry keys or values that have been changed
- Encryption operations performed by the malware

Future dynamic analysis tools may combine their capabilities with the following tools:

1. VirusTotal for multi-engine scanning and hash-based intelligence.
2. YARA rules to detect patterns of malware using different types of analytics.
3. Cuckoo Sandbox for automated dynamic analysis of malware.
4. ANY.RUN or other similar sandbox platforms for analyzing and producing behavior-based results.
5. Static methods of analyzing malware, such as strings, entropy, metadata, and file structure.
6. SIEM or other methods of analyzing logs, for correlation of suspicious activity.

By combining these tools with LLMs (Large Language Models), dynamic analysis will become more reliable than relying just on LLMs to classify malware based on visible code or encrypted data. Rather than the LLM being limited to determining a malware payload classification based only on visible code or encrypted data, a future application will include information organized into packets by other tools measuring both static and dynamic data. As such, the LLM could then determine the proof of its decision on legitimate evidence, such as Network Activity, Changes to the file system, Unusual API calls, Behavior of the payload in a sandbox environment.

As a result, it should significantly reduce the number of false positives and negatives generated by these methods. It would also help avoid cases where a large language model marks a

file as malicious just because it looks encrypted or has a lot of random data. By bringing together traditional malware analysis and reasoning based on large language models, future efforts can develop a more robust and evidence-driven framework for classifying malware.

Along with the malware samples, future analysis should include the non-malicious files such as normal scripts, encrypted text files, configuration files, and administrative tools. This would help the researchers to understand the positive behavior and determine whether LLMs incorrectly classify harmless opaque files as hostile.

10.6 Advanced Obfuscation and Transformation Techniques

For future enhancement, try out more advanced techniques for hiding and changing information. In this project, mainly used identifier renaming to hide information. Base92 encoding and AES encryption were used to help obscure the contents of the payload. These methods showed how large language models act when the readability of the content is lowered or hidden. In the experiments, it was observed that popular encoding formats like Base64 and Base85 were more easily recognized by the language models that were tested. When these usual encodings were used, the models had a better chance of recognizing or understanding the structure and labelling the payload as malicious. Base92 was chosen because it was harder for the tested models to recognize, and made it more challenging to interpret directly.

Future work can test stronger and more realistic transformation methods, such as:

1. Control-flow obfuscation helps to obscure the execution path.
2. String encryption allows for obfuscation of important strings such as file paths, commands, URLs, or suspicious function names.
3. Code packing or compression conceals the original structure of the payload.
4. Custom encoding methods are not as easy to detect as regular encoding methods.
5. Polymorphic transformation allows for changing the payload structure but retains the same functions.
6. Dead code insertion adds unnecessary code to confuse the analysis of the payload.
7. Function splitting and function merging split or merge code logic and make it difficult to interpret.

8. Multi-layer transformation provides an example of how obfuscation, custom encoding, compression, and encryption can be combined to protect the payload.

10.7 Multi-Language Expansion

In this project, several different payloads were tested with four different programming languages Python, Java, C++, and Shell scripting. This helped to determine if LLMs could identify similar malicious behaviors when expressed in different ways. Future experiments could be enhanced by testing additional programming languages frequently used to develop malware and/or security tools.

The future of project testing may include testing of new programming languages such as:

1. PowerShell: used primarily as an administrative tool in Windows environments, can also be used for creating scripts that exploit weaknesses in systems for malicious purposes.
2. JavaScript: a general-purpose scripting language that is commonly used for developing both web-based applications as well as browser-based payloads , JavaScript is typically one of the most widely used programming languages in the world and provides many opportunities for developers.
3. C#: an object-oriented programming language developed by Microsoft; can be deployed on Windows and .NET operating systems; often used to create malware on Windows systems.
4. Go (Golang): a programming language designed for cross-platform development; all cross-platform malware will eventually use Go (Golang) as its native language.
5. Rust: a systems programming language created to provide developers with an alternative way of developing software; Rust uses modern design patterns and is increasingly being used by malware authors to create new malware using memory-safe programming languages.
6. VBScript and Batch scripts: programming languages that were widely used to create malware on Windows systems were frequently targeted (and ultimately failed) for use against modern operating systems.

Future testing may also include testing new programming languages. This would allow for more accurate classification of malware based on how malware behaves and whether or not LLMs can distinguish malware from all other types of code, regardless of programming language.

By increasing the number of new languages tested, we will likely create a more accurate representation of how malware uses LLMs to classify malware across many different environments and operating systems. Malicious authors choose programming languages based on the platform they are targeting and the environment in which their code will run, as well as their need for stealth and the availability of required libraries. Therefore, increasing the number of languages being tested will allow us to understand how malware is classified using LLMs in a variety of environments and operating systems.

10.8 Summary

Possible improvements to the project in the future were discussed in this chapter. The present study was successful in evaluating the ability of various large language models (LLMs) to categorize simple, hidden, Base92 encoded, and AES encrypted data. The project can be improved by expanding the dataset, experimenting with more LLM platforms, improving automation, incorporating traditional malware analysis tools, and investigating more advanced transformation techniques.

Future work can have a dataset which has a broader range of samples which are specifically designed for evaluation. This dataset should include malicious, harmless, dual purpose, hidden, coded and encrypted samples. You can experiment with more LLM platforms and newer versions of models to see if the way we classify malware improves over time. A fully automated system using n8n or Python will help reduce manual effort and make the testing process more scalable and repeatable.

All of these, static analysis, dynamic analysis, sandboxing and tools to find old malware, can make the evaluation more fact and evidence based. Future research can also look into more complex ways of hiding and changing information, including control-flow obfuscation, string encryption, packing, custom encoding and polymorphic transformations.

Generally, such enhancements will enable more accurate project execution, allow the project to expand, and make it more like what we see in the "real world" of malware analysis. They will also help determine whether we can trust large language models (LLMs) as useful tools for malware identification.